

Министерство цифрового развития, связи и массовых коммуникаций РФ
ФГБОУ «Сибирский государственный университет телекоммуникаций и
информатики (СибГУТИ)
Уральский технический институт связи и информатики (филиал)
в г. Екатеринбурге (УрТИСИ СибГУТИ)

Кафедра информационных систем и
технологий

Учебная практика

МДК 02.02 Инструментальные средства
разработки программного обеспечения

Пояснительная записка

Студент Афанасьева А. А.

Группа 381

Министерство цифрового развития, связи и массовых коммуникаций РФ
ФГБОУ «Сибирский государственный университет телекоммуникаций и
информатики (СибГУТИ)

Уральский технический институт связи и информатики (филиал)
в г. Екатеринбурге (УрТИСИ СибГУТИ)

ОТЗЫВ

о работе обучающегося Афанасьевой Александры Андреевной

в период подготовки учебного проекта по теме «Разработка программного
модуля для учета заявок на ремонт бытовой техники.»

Направление подготовки: 09.02.07 Информационные системы и программирование

Преподаватель кафедры ИСТ
УрТИСИ

Преподаватель

_____ Бурумбаев Д. И.
(Бурумбаев Даниль Ильмирович)

« _____ » _____

Содержание

Введение.....	5
1 Анализ предметной области.....	6
2 Проектирование базы данных.....	8
2.1 Описание базы данных.....	8
2.2 ER-диаграмма.....	10
3 Разработка серверной части.....	11
3.1 Общая архитектура приложения.....	11
3.2 Реализация системы маршрутизации.....	11
3.3 Реализация бизнес-логики в представлениях.....	12
3.4 Система аутентификации и разграничения доступа.....	14
3.5 Реализация форм и валидации данных.....	15
3.6 Импорт данных из CSV-файлов.....	16
3.7 Описание логики работы приложения.....	16
4 Пользовательский интерфейс.....	18
4.1 Общее описание пользовательского интерфейса.....	18
4.2 Подробное описание пользовательского интерфейса.....	18
5 Описание дипломного проекта.....	32
5.1 Обоснование выбора инструментальных средств разработки.....	32
5.2 Структура приложения.....	32
5.3 Интеграция сайта с CRM-системой.....	49
Заключение.....	50
Библиография.....	51
Приложение А.....	52

Введение

Разработка современного веб-приложения для учёта и обработки заявок на ремонт бытовой техники позволяет значительно повысить эффективность работы сервисного центра. Процессы регистрации обращений клиентов, распределения заявок между мастерами и контроля выполнения ремонта, которые ранее могли выполняться вручную или с использованием различных таблиц и бумажных документов, автоматизированы с помощью единой информационной системы. Это позволяет сотрудникам быстрее обрабатывать заявки, контролировать их статус и своевременно выполнять ремонтные работы. Клиенты, в свою очередь, получают возможность удобно создавать и отслеживать свои заявки, что повышает уровень обслуживания и доверие к сервисному центру.

Данная тема является актуальной в связи с ростом количества бытовой техники и увеличением спроса на услуги сервисных центров. Эффективное управление заявками, контроль сроков выполнения работ и распределение нагрузки между мастерами требуют использования современных информационных систем. Использование веб-приложения позволяет централизовать хранение данных, повысить прозрачность процессов и снизить вероятность ошибок при обработке информации.

Целью учебной практики является разработка веб-приложения для автоматизации работы сервисного центра по ремонту бытовой техники.

Для достижения поставленной цели необходимо решить следующие задачи:

- разработать пользовательский веб-интерфейс для создания и просмотра заявок на ремонт;
- реализовать серверную часть приложения для обработки запросов и взаимодействия с базой данных;
- обеспечить хранение и обработку информации о пользователях, заявках и комментариях;
- реализовать систему ролей пользователей и управление доступом к функционалу приложения;
- обеспечить корректную и стабильную работу всех модулей системы;
- подготовить пояснительную записку по выполненной работе.

Обоснование выбора темы курсового проекта

1 Анализ предметной области

Сервисный центр «БытСервис» занимается приёмом, регистрацией и выполнением заявок на ремонт бытовой техники. В перечень обслуживаемого оборудования входят холодильники, стиральные машины, плиты, микроволновые печи, мультиварки, фены, тостеры и другие виды бытовой техники. Для повышения качества обслуживания клиентов, сокращения времени обработки обращений и упрощения работы сотрудников разработана информационная система для учёта заявок на ремонт.

Основные бизнес-процессы сервисного центра включают приём заявок от клиентов по телефону или через веб-интерфейс, регистрацию заявки с указанием типа техники, модели и описания проблемы, назначение мастера для выполнения ремонта, выполнение ремонтных работ с фиксацией использованных запчастей, информирование клиента о готовности техники, контроль качества выполненных работ и сбор обратной связи. Каждый из этих процессов требует тщательного учёта и координации между различными сотрудниками: операторами, принимающими заявки, мастерами, выполняющими ремонт, и менеджерами, контролирующими общие показатели работы.

В ходе анализа предметной области были выявлены следующие проблемы существующей системы работы. Отсутствие единой базы данных приводит к дублированию информации и ошибкам при передаче данных между сотрудниками, информация о заявках хранится в разных местах, что затрудняет поиск и анализ. Сложность отслеживания текущего статуса заявки не позволяет оперативно отвечать на запросы клиентов о ходе выполнения ремонта. Затруднён контроль загрузки мастеров, что может приводить к неравномерному распределению нагрузки и задержкам в выполнении работ. Отсутствие статистики не даёт возможности анализировать эффективность работы сервисного центра, выявлять наиболее частые неисправности и планировать закупку запчастей. Невозможность для клиента самостоятельно отслеживать статус своей заявки снижает уровень удовлетворённости обслуживанием и увеличивает нагрузку на операторов, вынужденных отвечать на однотипные вопросы.

Разработанный программный модуль предназначен для автоматизации основных процессов сервисного центра и обеспечивает следующий функционал. Регистрация новых заявок на ремонт с указанием типа техники, модели и описания проблемы позволяет быстро и точно фиксировать все необходимые данные. Просмотр списка заявок с возможностью поиска по ФИО клиента даёт сотрудникам удобный доступ к информации. Редактирование информации о заявках позволяет корректировать данные при необходимости. Изменение статуса выполнения заявки отражает текущий этап работы: новая заявка, в процессе ремонта или готова к выдаче. Назначение мастеров на выполнение работ обеспечивает распределение нагрузки. Добавление комментариев и сведений о запчастях фиксирует историю ремонта. Продление срока выполнения заявки с обязательным согласованием с клиентом учитывает возможные задержки. Просмотр статистики по выполненным заявкам даёт менеджерам инструмент для

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		6

анализа. Генерация QR-кода для перехода к форме оценки качества обслуживания позволяет собирать обратную связь.

В результате автоматизации достигаются следующие преимущества. Сокращается время на обработку заявок благодаря единой базе данных и быстрому доступу к информации. Повышается прозрачность процессов для всех участников: сотрудники видят актуальные данные, клиенты могут отслеживать статус. Снижается вероятность ошибок при вводе и передаче данных за счёт автоматической валидации и исключения ручного дублирования. Появляется возможность анализа загрузки мастеров и эффективности работы на основе статистических данных. Повышается удовлетворённость клиентов за счёт информирования о статусе заявок и возможности оставить отзыв.

Используемый стек технологий включает:

- Python версии 3.10 как основной язык программирования;
- Django версии 4.2 в качестве веб-фреймворка;
- HTML, CSS и Bootstrap 5 для создания пользовательского интерфейса;
- SQLite как систему управления базами данных;
- библиотеку qrcode для генерации QR-кодов;

Выбор Django обусловлен его богатым функционалом, включающим встроенную систему аутентификации, ORM для работы с базами данных, систему маршрутизации, шаблонизатор и административную панель. SQLite выбрана из-за простоты развёртывания и отсутствия необходимости в отдельном сервере, что оптимально для данного проекта.

					09.02.07.000023 П.218 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

2 Проектирование базы данных

2.1 Описание базы данных

В качестве системы управления базами данных использовалась SQLite, которая является встроенной реляционной базой данных и не требует отдельного сервера. Выбор обусловлен простотой развёртывания, достаточной производительностью для данного проекта и отсутствием необходимости в сложном администрировании. База данных предназначена для хранения информации о пользователях, заявках и комментариях.

Таблицы базы данных:

1. таблица «Пользователи» (users_user)

Содержит сведения обо всех пользователях системы. Таблица создана на основе кастомной модели пользователя Django и включает следующие поля:

- id (INTEGER, PRIMARY KEY) – уникальный идентификатор пользователя, автоматически инкрементируемое значение;
- username (VARCHAR) – логин для входа в систему, уникальное значение;
- password (VARCHAR) – хэш пароля в зашифрованном виде;
- fio (VARCHAR) – полное ФИО пользователя;
- phone (VARCHAR) – номер телефона для связи;
- user_type (VARCHAR) – роль пользователя в системе;
- email (VARCHAR) – электронная почта;
- is_staff (BOOLEAN) – флаг, указывающий, является ли пользователь сотрудником;
- date_joined (DATETIME) – дата и время регистрации пользователя.

Поле user_type может принимать следующие значения, определяющие роль пользователя и его права доступа:

- Администратор – полный доступ ко всем функциям системы;
- Оператор – создание, редактирование и удаление заявок;
- Мастер – просмотр заявок и добавление комментариев;
- Менеджер – просмотр статистики и заявок без возможности изменения;
- Менеджер по качеству – продление срока, назначение мастеров, консультации;
- Заказчик – просмотр только своих заявок и согласование продления.

2. Таблица «Заявки» (requests_request)

Содержит информацию о всех заявках на ремонт бытовой техники, поступающих в сервисный центр. Таблица включает следующие поля:

- requestID (INTEGER, PRIMARY KEY) – уникальный идентификатор заявки, автоматически инкрементируемое значение;
- startDate (DATE) – дата регистрации заявки, устанавливается автоматически при создании;
- homeTechType (VARCHAR) – вид бытовой техники, требующей ремонта;
- homeTechModel (VARCHAR) – модель техники;
- problemDescription (TEXT) – подробное описание неисправности со слов

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		8

клиента;

- requestStatus (VARCHAR) – текущий статус заявки;
- completionDate (DATE) – дата фактического завершения ремонта, может быть NULL;
- repairParts (TEXT) – информация об использованных запчастях и материалах, может быть NULL;
- deadline_extended (BOOLEAN) – флаг, указывающий, было ли запрошено продление срока;
- deadline_extended_date (DATE) – новая планируемая дата завершения при продлении срока;
- extension_approved (BOOLEAN) – флаг, отражающий факт согласования продления с клиентом;
- client_id (INTEGER, FOREIGN KEY REFERENCES users_user(id)) – идентификатор клиента, создавшего заявку;
- master_id (INTEGER, FOREIGN KEY REFERENCES users_user(id)) – идентификатор назначенного мастера, может быть NULL.

Поле homeTechType может принимать следующие значения:

- Холодильник;
- Стиральная машина;
- Плита;
- Микроволновка;
- Мультиварка;
- Фен;
- Тостер.

Поле requestStatus может принимать следующие значения, отражающие этапы выполнения работ:

- Новая заявка – только создана, мастер не назначен;
- В процессе ремонта – назначен мастер, ведутся работы;
- Готова к выдаче – ремонт завершён, техника ожидает клиента.

3. Таблица «Комментарии» (requests_comment)

Содержит комментарии, оставляемые мастерами и менеджерами к заявкам. Реализует двухуровневую систему коммуникации между сотрудниками и клиентами. Таблица включает следующие поля:

- commentID (INTEGER, PRIMARY KEY) – уникальный идентификатор комментария, автоматически инкрементируемое значение;
- message (TEXT) – текст комментария;
- created_at (DATETIME) – дата и время создания комментария, устанавливается автоматически;
- comment_type (VARCHAR) – тип комментария;
- master_id (INTEGER, FOREIGN KEY REFERENCES users_user(id)) – идентификатор пользователя, оставившего комментарий;
- request_id (INTEGER, FOREIGN KEY REFERENCES requests_request(requestID)) – идентификатор заявки, к которой относится комментарий.

Поле `comment_type` может принимать следующие значения:

- `internal` – внутренний комментарий, видимый только сотрудникам;
- `client` – сообщение для клиента, отображаемое заказчику.

Структура базы данных обеспечивает целостность информации и отражает реальные отношения между сущностями предметной области:

- каждая заявка связана с клиентом через внешний ключ `client_id`, что позволяет определить, кто именно обратился за ремонтом;
- каждая заявка может быть связана с мастером через внешний ключ `master_id`, что отражает назначение ответственного исполнителя;
- один пользователь может быть связан с несколькими заявками как клиент, что соответствует ситуациям, когда один заказчик обращается несколько раз;
- один пользователь может быть связан с несколькими заявками как мастер, что отражает его загрузку и участие в разных ремонтах;
- одна заявка может содержать несколько комментариев, что позволяет вести историю обсуждения хода работ;
- один пользователь может оставить несколько комментариев, что фиксирует его активность в системе.

Таким образом, спроектированная структура базы данных позволяет реализовать полный цикл сопровождения заявки: от регистрации обращения клиента до завершения ремонта, информирования заказчика и последующего анализа качества работы. Для первоначального наполнения базы данных использовались CSV-файлы, предоставленные заказчиком, которые были импортированы с помощью специально разработанной Django-команды `import_csv`.

2.2 ER-диаграмма

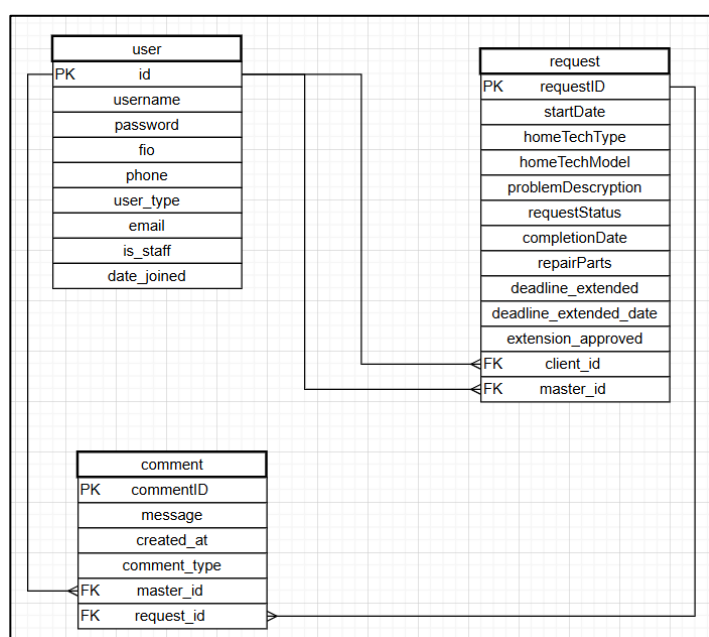


Рисунок 2.1 – ER-диаграмма базы данных

3 Разработка серверной части

3.1 Общая архитектура приложения

Серверная часть программного модуля реализована на базе Django — высокоуровневого веб-фреймворка на Python, который следует архитектурному шаблону MVT (Model-View-Template). Выбор Django обусловлен его богатым функционалом, включающим встроенную систему аутентификации, ORM для работы с базами данных, систему маршрутизации, шаблонизатор и административную панель.

Django используется для выполнения следующих задач:

- обработки GET- и POST-запросов от клиентов;
- маршрутизации между различными страницами приложения;
- обработки форм авторизации, создания и редактирования заявок;
- взаимодействия с базой данных через встроенный ORM;
- передачи данных из базы в HTML-шаблоны;
- аутентификации пользователей и разграничения прав доступа.

Подключение к базе данных выполняется автоматически через настройки Django, описанные в файле `settings.py`. Взаимодействие с базой данных осуществляется с помощью ORM Django, что позволяет выполнять операции создания, чтения, обновления и удаления записей без написания прямых SQL-запросов. Это упрощает разработку и обеспечивает безопасность приложения.

Приложение разделено на два основных модуля для соблюдения принципа разделения ответственности:

- модуль `users` отвечает за работу с пользователями, их аутентификацию и управление ролями;
- модуль `requests` реализует всю логику работы с заявками, комментариями и статистикой.

Такое разделение обеспечивает модульность, упрощает поддержку и тестирование кода, а также позволяет при необходимости расширять функциональность, добавляя новые модули.

3.2 Реализация системы маршрутизации

В приложении реализованы следующие основные маршруты, обеспечивающие навигацию и функциональность для различных категорий пользователей:

Маршруты для аутентификации и управления сессией:

- `path("", views.login_view, name='login')` – главная страница, отображает форму входа в систему. При отправке формы проверяет логин и пароль, при успешной аутентификации перенаправляет пользователя на дашборд;

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		11

– path('logout/', views.logout_view, name='logout') – завершает сеанс пользователя и перенаправляет на страницу входа.

Маршруты для панели управления:

– path('dashboard/', views.dashboard, name='dashboard') – главная страница после входа, отображает панель управления с адаптацией под роль пользователя. Для заказчика показывает список его заявок, для сотрудников – статистику и последние заявки.

Маршруты для работы с заявками:

– path('requests/', views.request_list, name='request_list') – отображает список всех заявок с возможностью поиска по ФИО клиента;

– path('requests/create/', views.request_create, name='request_create') – форма создания новой заявки, доступна только операторам и администраторам;

– path('requests/<int:pk>/', views.request_detail, name='request_detail') – детальный просмотр конкретной заявки с отображением всей информации и комментариев;

– path('requests/<int:pk>/update/', views.request_update, name='request_update') – редактирование заявки, доступно операторам и администраторам;

– path('requests/<int:pk>/delete/', views.request_delete, name='request_delete') – удаление заявки с предварительным подтверждением через модальное окно.

Маршруты для дополнительного функционала:

– path('requests/<int:pk>/assign/', views.assign_master, name='assign_master') – назначение мастера на заявку, доступно операторам, администраторам и менеджерам по качеству;

– path('requests/<int:pk>/extend/', views.extend_deadline, name='extend_deadline') – отправка запроса на продление срока, доступно только менеджеру по качеству;

– path('requests/<int:pk>/approve/', views.approve_extension, name='approve_extension') – согласование или отклонение продления срока, доступно только заказчику, создавшему заявку;

– path('requests/<int:pk>/qr/', views.generate_qr, name='generate_qr') – генерация QR-кода со ссылкой на форму обратной связи.

3.3 Реализация бизнес-логики в представлениях

Бизнес-логика приложения реализована в функциях-представлениях, каждая из которых выполняет определённый набор операций в зависимости от назначения.

Функция login_view обрабатывает аутентификацию пользователей:

– при GET-запросе отображает страницу входа с формой для ввода логина и пароля;

– при POST-запросе извлекает из формы логин и пароль;

– вызывает встроенную функцию authenticate для проверки учётных данных;

– при успешной аутентификации функция login создаёт сессию пользователя;

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		12

- пользователь перенаправляется на дашборд;
- при ошибке отображается сообщение о неверном логине или пароле.

Функция `dashboard` является центральным представлением после входа и выполняет различные действия в зависимости от роли пользователя:

Для заказчика:

- выполняет запрос к базе данных, выбирая все заявки, где `client` равен текущему пользователю;
- передаёт полученный список в шаблон для отображения.

Для сотрудников:

- получает последние 10 заявок с помощью `Request.objects.order_by('-startDate')[:10]`;
- рассчитывает общее количество заявок через `Request.objects.count()`;
- подсчитывает количество заявок по каждому статусу с использованием фильтрации;

– рассчитывает среднее время ремонта: выбираются все завершённые заявки со статусом «Готова к выдаче» и заполненной датой завершения, для каждой вычисляется разница между `completionDate` и `startDate`, полученные значения суммируются и делятся на количество заявок;

– формирует топ неисправностей через группировку по `homeTechType` с подсчётом количества;

– формирует динамику по месяцам с использованием функции `TruncMonth`;

– передаёт все рассчитанные показатели в шаблон через контекст.

Функция `request_detail` обеспечивает детальный просмотр заявки:

– получает заявку по переданному идентификатору или возвращает ошибку 404;

– проверяет, имеет ли пользователь доступ к данной заявке: если пользователь является заказчиком, но заявка принадлежит другому клиенту, выводится сообщение об ошибке и выполняется перенаправление;

– в зависимости от роли пользователя формирует набор комментариев: для заказчика фильтруются только комментарии с типом 'client', для сотрудников показываются все комментарии;

– при POST-запросе от пользователя с ролью мастера или менеджера по качеству создаётся новый комментарий с указанием типа;

– после сохранения комментария пользователь перенаправляется обратно на страницу детального просмотра.

Функция `request_create` реализует создание новых заявок:

– проверяет, имеет ли пользователь права на создание заявок (роль оператора или администратора);

– при GET-запросе отображает пустую форму для заполнения;

– при POST-запросе передаёт полученные данные в форму для валидации;

– если данные корректны, создаётся новый объект `Request` с установкой `startDate` равной текущей дате и `requestStatus` «Новая заявка»;

– после сохранения пользователь видит сообщение об успешном создании и перенаправляется на страницу созданной заявки;

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		13

- при наличии ошибок валидации они отображаются пользователю с пояснениями.

Функция `extend_deadline` реализует механизм продления срока:

- проверяет, что пользователь имеет роль «Менеджер по качеству»;
- при POST-запросе извлекает из формы новую дату и комментарий;
- устанавливает флаги `deadline_extended = True` и `extension_approved = False`;
- сохраняет новую дату в поле `deadline_extended_date`;
- создаёт комментарий типа 'client' для уведомления клиента;
- выводит сообщение об отправке запроса и перенаправляет на страницу заявки.

Функция `approve_extension` предоставляет заказчику возможность ответить на запрос:

- проверяет, что пользователь имеет роль «Заказчик» и заявка принадлежит ему;
- при POST-запросе в зависимости от выбранного действия устанавливает соответствующие флаги;
- создаёт комментарий, фиксирующий решение клиента;
- выводит сообщение о результате и перенаправляет на страницу заявки.

Функция `generate_qr` создаёт QR-код для оценки качества:

- получает заявку по идентификатору;
- создаёт объект `QRCode` с параметрами `version=1`, `box_size=10`, `border=5`;
- добавляет в QR-код ссылку на Google-форму для сбора обратной связи;
- генерирует изображение с чёрно-белой цветовой схемой;
- сохраняет изображение в оперативную память через `BytesIO`;
- кодирует изображение в формат `base64` для встраивания в HTML;
- передаёт закодированное изображение в шаблон;
- QR-код отображается на странице детального просмотра заявки.

3.4 Система аутентификации и разграничения доступа

В проекте используется встроенная система аутентификации Django, расширенная кастомной моделью пользователя. В файле `settings.py` указана настройка `AUTH_USER_MODEL = 'users.User'`, что позволяет использовать созданную модель вместо стандартной.

Разграничение прав доступа реализовано на нескольких уровнях для обеспечения надёжной защиты:

На уровне маршрутов проверка выполняется в каждой функции-представлении:

- проверяется, аутентифицирован ли пользователь через `request.user.is_authenticated`;
- проверяется тип пользователя через `request.user.user_type`;
- при отсутствии прав выводится сообщение об ошибке через `messages.error`;

- выполняется перенаправление на дашборд.

На уровне шаблонов с помощью условных операторов языка шаблонов Django:

- отображаются или скрываются кнопки создания заявок для операторов;
- отображаются или скрываются кнопки редактирования и удаления;
- отображаются или скрываются кнопки продления срока для менеджера по качеству;
- отображаются или скрываются кнопки согласования для заказчика.

На уровне данных при запросах к базе учитывается роль пользователя:

- заказчик видит только заявки, где `client = request.user`;
- сотрудники видят все заявки без ограничений;
- комментарии фильтруются в зависимости от роли: заказчик видит только комментарии с типом 'client'.

3.5 Реализация форм и валидации данных

Для работы с формами используется встроенный механизм Django Forms, который обеспечивает генерацию HTML-кода форм, валидацию введенных данных и их преобразование в Python-объекты. В файле `forms.py` определены две основные формы.

`RequestForm` предназначена для создания и редактирования заявок и включает следующие поля:

- `homeTechType` – выпадающий список с предопределёнными вариантами типов техники;
- `homeTechModel` – текстовое поле для ввода модели;
- `problemDescription` – многострочное текстовое поле для описания проблемы;
- `client` – выпадающий список с пользователями типа «Заказчик»;
- `master` – выпадающий список с пользователями типа «Мастер», необязательное поле.

`CommentForm` предназначена для добавления комментариев и включает следующие поля:

- `message` – многострочное текстовое поле для ввода комментария;
- `comment_type` – выпадающий список для выбора типа комментария (`internal` или `client`), отображается только для менеджера по качеству.

Валидация данных выполняется автоматически на основе типов полей модели, ограничений, заданных в модели, и дополнительных проверок, заданных в форме. При возникновении ошибок валидации форма сохраняет введенные пользователем данные, а ошибки отображаются рядом с соответствующими полями.

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		15

3.6 Импорт данных из CSV-файлов

Для первоначального наполнения базы данных разработана специальная management-команда `import_csv`, расположенная в файле `import.py`. Команда последовательно обрабатывает три файла, предоставленных заказчиком.

При импорте пользователей из файла `users.csv`:

- открывается файл с кодировкой UTF-8;
- читаются данные с разделителем точка с запятой;
- для каждой строки создаётся объект `User` с указанным идентификатором;
- заполняются поля `username`, `fio`, `phone`, `user_type`;
- пароль хэшируется с помощью `make_password`;
- пользователь сохраняется в базу данных.

При импорте заявок из файла `requests.csv`:

- для каждой строки находится клиент по идентификатору `clientID`;
- при наличии `master_id` находится соответствующий мастер;
- при наличии `completionDate` преобразуется в формат `date`;
- `startDate` преобразуется в формат `date`;
- создаётся объект `Request` с заполнением всех полей;
- заявка сохраняется в базу данных.

При импорте комментариев из файла `comments.csv`:

- для каждой строки находится мастер по идентификатору `masterID`;
- находится заявка по идентификатору `requestID`;
- создаётся объект `Comment` с заполнением полей;
- комментарий сохраняется в базу данных.

Команда может быть запущена из командной строки: `python manage.py import.py`. Это обеспечивает быстрое развёртывание системы с тестовыми данными.

3.7 Описание логики работы приложения

После запуска приложения пользователь попадает на страницу авторизации, где вводит свой логин и пароль. Система проверяет учётные данные и определяет роль пользователя. В зависимости от роли пользователь перенаправляется на соответствующую панель управления.

Для заказчика (клиента) доступен следующий функционал:

- просмотр списка своих заявок с отображением текущего статуса каждой;
- детальный просмотр любой своей заявки с возможностью увидеть комментарии, адресованные клиенту;
- получение уведомлений о запросе на продление срока;
- согласование или отклонение запроса на продление с помощью соответствующих кнопок;

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		16

– просмотр QR-кода для оценки качества обслуживания.

Для оператора доступен следующий функционал:

– просмотр полного списка всех заявок с возможностью поиска по ФИО клиента;

– создание новых заявок с автоматической простановкой текущей даты;

– редактирование любых заявок, включая изменение типа техники, модели и описания;

– удаление заявок с обязательным подтверждением через модальное окно;

– назначение мастеров на выполнение работ;

– просмотр детальной информации по каждой заявке.

Для мастера доступен следующий функционал:

– просмотр всех заявок, включая заявки других мастеров;

– добавление комментариев к заявкам с описанием выполненных работ;

– фиксация использованных запчастей в тексте комментариев;

– отсутствие прав на редактирование и удаление заявок;

– просмотр комментариев, оставленных другими мастерами и менеджерами.

Для менеджера по качеству доступен следующий функционал:

– просмотр всех заявок и статистики работы сервисного центра;

– назначение мастеров на заявки при необходимости перераспределения нагрузки;

– отправка запросов на продление срока с указанием новой даты и причины;

– добавление комментариев двух типов: внутренних для сотрудников и сообщений для клиента;

– консультирование мастеров при возникновении сложных ситуаций;

– контроль соблюдения сроков выполнения работ.

Для менеджера доступен следующий функционал:

– просмотр всех заявок без возможности их изменения;

– анализ статистических данных о работе сервисного центра;

– отслеживание загрузки мастеров и эффективности их работы;

– формирование отчётов о количестве выполненных заявок.

Для получения оценки качества обслуживания в системе предусмотрена генерация QR-кода. При нажатии на кнопку «ПОЛУЧИТЬ QR-КОД» на странице детального просмотра заявки генерируется изображение, содержащее ссылку на Google-форму для сбора обратной связи. Клиент может отсканировать код камерой своего телефона и перейти к форме опроса, что позволяет сервисному центру собирать отзывы о качестве выполненных работ и улучшать обслуживание.

4 Пользовательский интерфейс

4.1 Общее описание пользовательского интерфейса

Пользовательский интерфейс реализован в виде веб-приложения с использованием HTML, CSS и встроенного шаблонизатора Django. Интерфейс выполнен в строгом темном дизайне с единым стилем для всех страниц, что обеспечивает комфортную работу пользователей и визуальную согласованность всех элементов. При разработке интерфейса особое внимание уделялось простоте использования, интуитивной понятности и адаптивности под различные устройства.

Основные элементы интерфейса включают:

- навигационную панель;
- карточки статистики с крупными цифрами;
- таблицы для отображения списков заявок;
- формы для ввода и редактирования данных;
- кнопки с иконками для основных операций;
- модальные окна для подтверждения действий;
- цветовые индикаторы статусов заявок;

Все страницы имеют единую структуру: в верхней части располагается навигационная панель с названием приложения «БЫТСЕРВИС» и заявки, в центральной части отображается основное содержимое, предусмотрены кнопки для возврата к предыдущим страницам. Навигация и доступный функционал зависят от роли пользователя, что обеспечивает безопасность и удобство работы.

4.2 Подробное описание пользовательского интерфейса

1. Страница авторизации.

Страница авторизации является начальной страницей приложения и доступна по корневому URL. На ней располагается форма входа в систему, выполненная в виде карточки с затемненным фоном и четкими контурами.

Форма входа содержит следующие элементы:

- поле для ввода логина с подсказкой «Логин»;
- поле для ввода пароля с подсказкой «Пароль»;
- кнопка «ВОЙТИ» синего цвета для отправки формы;
- информационный блок с перечнем доступных ролей и их прав.

При вводе неверных данных отображается всплывающее сообщение об ошибке красного цвета с текстом «Неверный логин или пароль». После трех секунд сообщение автоматически исчезает. При успешной авторизации пользователь перенаправляется на панель управления, соответствующую его роли.

Дизайн страницы выполнен в темных тонах с контрастными элементами: фон

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		18

страницы темно-серый, карточка формы имеет более светлый оттенок, поля ввода подсвечиваются при фокусировке. Это создает строгий и профессиональный внешний вид, соответствующий корпоративному стилю.

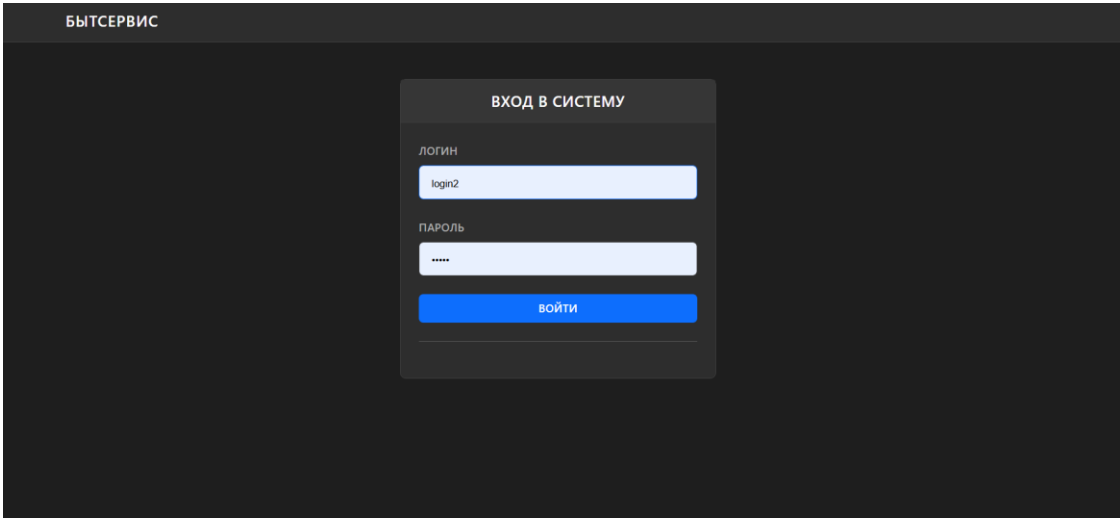


Рисунок 4.1 – Страница авторизации

2. Панель управления (дашборд).

Панель управления является главной страницей после авторизации и динамически адаптируется под роль пользователя. В верхней части отображается имя пользователя, при нажатии на него можно увидеть роль и выйти из профиля.

Для заказчика на панели управления отображаются:

- таблица со списком всех обращений клиента;
- для каждой заявки выводятся дата, тип техники, модель, статус и мастер;
- статусы выделяются цветными бейджами: новые – желтым, в работе – синим, готовые – зеленым;
- кнопка «ПРОСМОТР» для перехода к детальной информации.

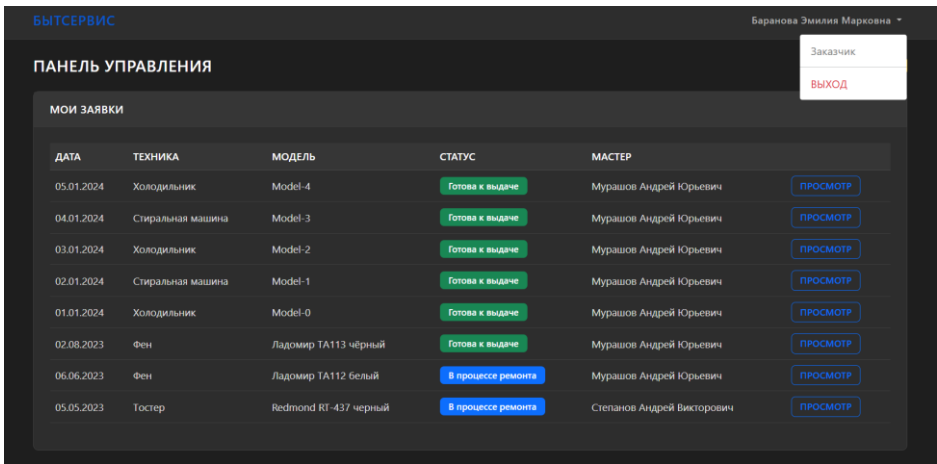


Рисунок 4.2 – Панель управления для заказчика

Для сотрудников (операторов, мастеров, менеджеров) на панели управления отображаются:

- карточки с основными статистическими показателями;

- таблица «ПОСЛЕДНИЕ ЗАЯВКИ» с десятью последними обращениями;
- для каждой заявки выводятся дата, клиент, телефон, техника, модель, статус, мастер;
- кнопка «ВСЕ ЗАЯВКИ» для перехода к полному списку;
- для менеджеров дополнительно отображается детальная статистика.

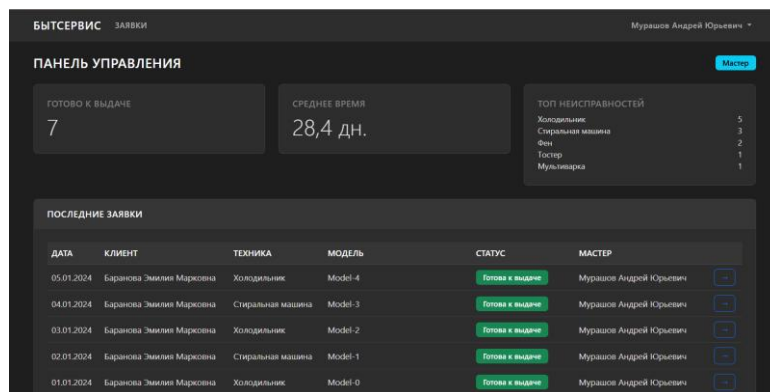


Рисунок 4.3 – Панель управления для сотрудника

Для менеджера и менеджера по качеству на панели управления дополнительно отображаются:

- расширенные статистические карточки с детальными показателями;
- блок «СТАТУСЫ ЗАЯВОК» с распределением по статусам;
- блок «ТОП НЕИСПРАВНОСТЕЙ» с рейтингом наиболее частых поломок;
- блок «ДИНАМИКА ПО МЕСЯЦАМ» с таблицей количества заявок.

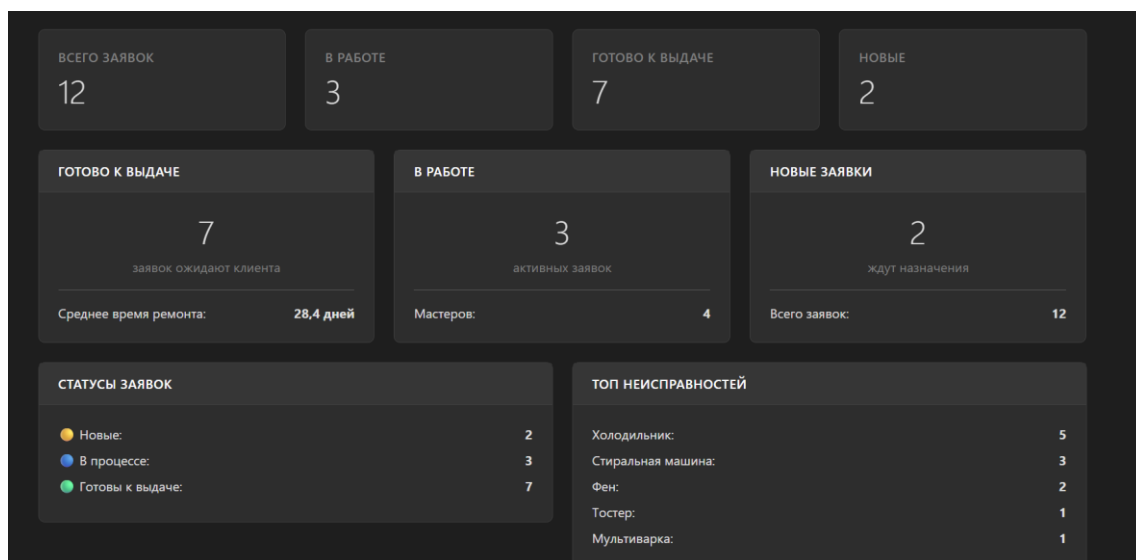


Рисунок 4.4 – Панель управления для менеджера с расширенной статистикой

3. Страница списка заявок.

Страница списка заявок доступна по URL /requests/ и предназначена для просмотра и управления всеми заявками в системе. Страница имеет единую структуру для всех сотрудников, но набор доступных действий зависит от роли

пользователя.

- В верхней части страницы расположены:
- заголовок «УПРАВЛЕНИЕ ЗАЯВКАМИ»;
 - для операторов и администраторов кнопка «НОВАЯ ЗАЯВКА» синего цвета;
 - блок поиска с полем для ввода ФИО клиента и кнопкой «НАЙТИ».

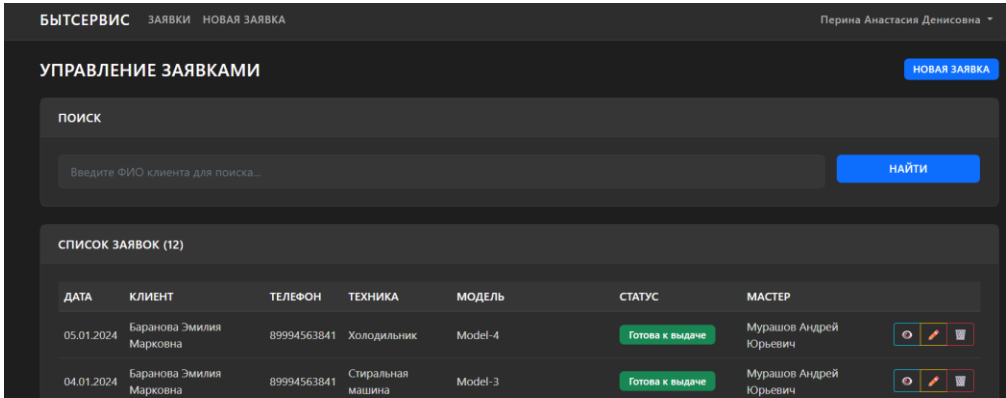


Рисунок 4.5 – Верхняя часть страницы списка заявок с поиском

Основную часть страницы занимает таблица со списком заявок, содержащая следующие колонки:

- ДАТА – дата создания заявки в формате дд.мм.гггг;
- КЛИЕНТ – фамилия, имя и отчество клиента;
- ТЕЛЕФОН – контактный номер телефона;
- ТЕХНИКА – тип бытовой техники;
- МОДЕЛЬ – модель техники;
- СТАТУС – текущий статус заявки с цветовым индикатором;
- МАСТЕР – фамилия назначенного мастера или прочерк;
- ДЕЙСТВИЯ – набор кнопок для операций с заявкой.

СПИСОК ЗАЯВОК (12)							
ДАТА	КЛИЕНТ	ТЕЛЕФОН	ТЕХНИКА	МОДЕЛЬ	СТАТУС	МАСТЕР	
05.01.2024	Баранова Эмилия Марковна	89994563841	Холодильник	Model-4	Готова к выдаче	Мурашов Андрей Юрьевич	  
04.01.2024	Баранова Эмилия Марковна	89994563841	Стиральная машина	Model-3	Готова к выдаче	Мурашов Андрей Юрьевич	  
03.01.2024	Баранова Эмилия Марковна	89994563841	Холодильник	Model-2	Готова к выдаче	Мурашов Андрей Юрьевич	  
02.01.2024	Баранова Эмилия Марковна	89994563841	Стиральная машина	Model-1	Готова к выдаче	Мурашов Андрей Юрьевич	  
01.01.2024	Баранова Эмилия Марковна	89994563841	Холодильник	Model-0	Готова к выдаче	Мурашов Андрей Юрьевич	  
02.08.2023	Егорова Алиса Платоновна	89994563842	Стиральная машина	DEXP WM-F610NTMA/VWV белый	Новая заявка	—	  
02.08.2023	Титов Максим Иванович	89994563843	Мультиварка	Redmond RMC-M95 черный	Новая заявка	—	  
02.08.2023	Баранова Эмилия Марковна	89994563841	Фен	Ладомир TA113 чёрный	Готова к выдаче	Мурашов Андрей Юрьевич	  
09.07.2023	Егорова Алиса Платоновна	89994563842	Холодильник	Indesit DS 314 W серый	В процессе ремонта	Степанов Андрей Викторович	  

Рисунок 4.6 – Таблица со списком заявок

Для операторов и администраторов в колонке действий доступны три кнопки:

- кнопка с иконкой глаза для просмотра детальной информации;
- кнопка с иконкой карандаша для редактирования заявки;
- кнопка с иконкой корзины для удаления заявки с подтверждением.

Для мастеров и менеджеров доступна только кнопка просмотра. При отсутствии заявок отображается соответствующее информационное сообщение.

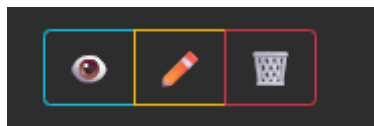


Рисунок 4.7 – Кнопки действий для оператора

4. Модальное окно подтверждения удаления.

При нажатии на кнопку удаления открывается модальное окно с подтверждением действия. Окно содержит предупреждение о необратимости операции и две кнопки: «ОТМЕНА» и «УДАЛИТЬ». Такой механизм предотвращает случайное удаление важных данных.

Модальное окно содержит следующие элементы:

- заголовок «ПОДТВЕРЖДЕНИЕ»;
- текст с вопросом об удалении конкретной заявки;
- предупреждение «Действие необратимо» красным цветом;
- кнопка «ОТМЕНА» серого цвета для закрытия окна;
- кнопка «УДАЛИТЬ» красного цвета для подтверждения.

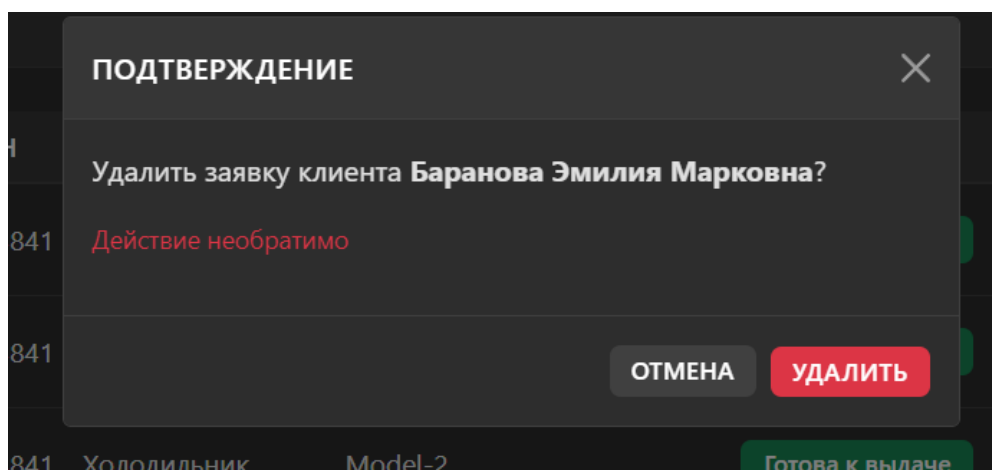


Рисунок 4.8 – Модальное окно подтверждения удаления

5. Страница детального просмотра заявки.

Страница детального просмотра заявки доступна по URL /requests/<id>/ и отображает полную информацию о конкретном обращении. Страница разделена на две колонки: основную с информацией о заявке и комментариями, и боковую с данными о клиенте и мастере.

В верхней части страницы расположены:

- кнопка «← НАЗАД» для возврата к предыдущей странице;

					09.02.07.000023 П.218 ПЗ	Лист
						22
Изм.	Лист	№ докум.	Подпись	Дата		

- заголовок с номером заявки;
- набор кнопок действий в зависимости от роли пользователя.

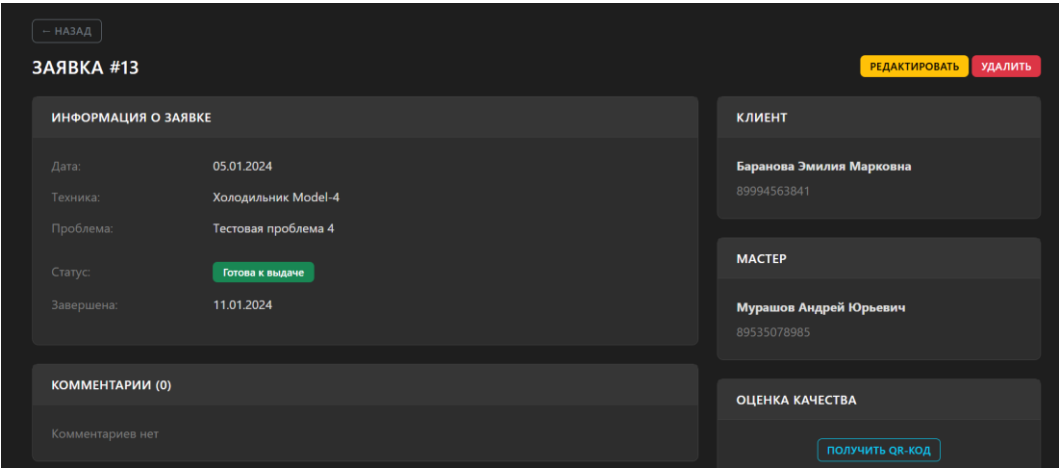


Рисунок 4.9 –Страница детального просмотра

Основная колонка содержит следующие блоки:

- блок «ИНФОРМАЦИЯ О ЗАЯВКЕ» с полным описанием: дата создания, тип техники, модель, описание проблемы, статус, дата завершения, информация о продлении срока;
- блок «КОММЕНТАРИИ» со списком всех комментариев к заявке, отсортированных по дате создания.

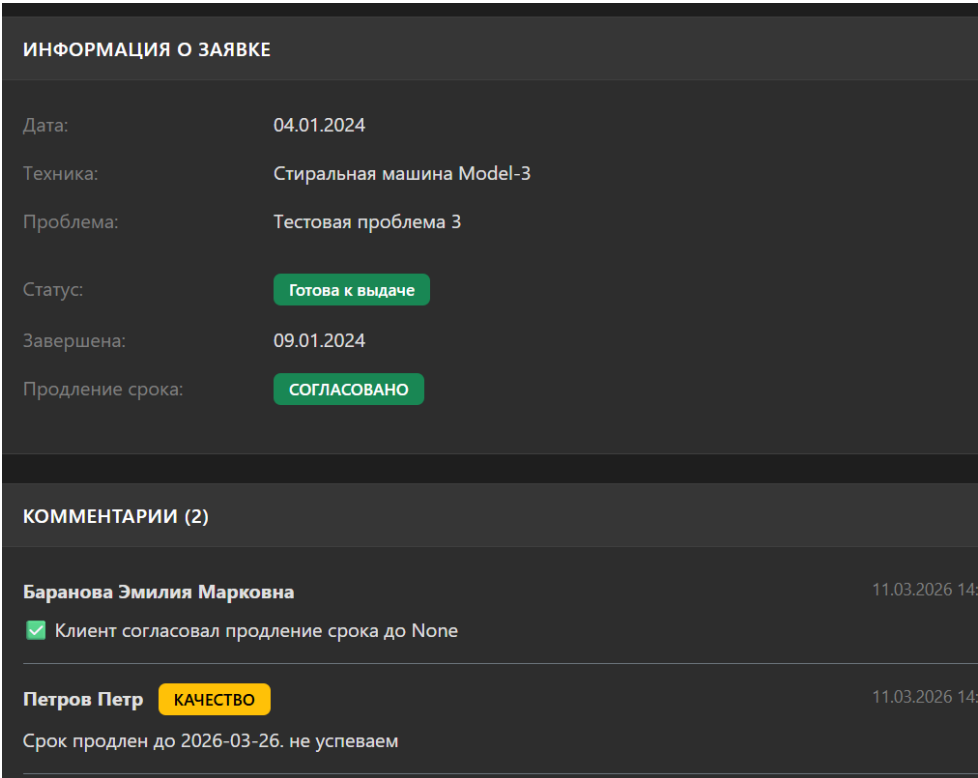


Рисунок 4.10 – Блок информации о заявке

Боковая колонка содержит следующие блоки:

- блок «КЛИЕНТ» с фамилией и телефоном заказчика;
- блок «МАСТЕР» с информацией о назначенном мастере или формой для назначения;
- блок «ОЦЕНКА КАЧЕСТВА» с кнопкой генерации QR-кода или самим QR-кодом.

КЛИЕНТ

Баранова Эмилия Марковна
89994563841

МАСТЕР

Мурашов Андрей Юрьевич
89535078985

ОЦЕНКА КАЧЕСТВА



Отсканируйте для оценки работы сервиса

Рисунок 4.11 – Боковая колонка с информацией о клиенте и мастере

Для мастеров и менеджеров по качеству в блоке комментариев доступна кнопка «+» для добавления нового комментария. При нажатии открывается форма с полем для ввода текста и для менеджера по качеству дополнительным полем выбора типа комментария.

КОММЕНТАРИИ (0)

Введите комментарий...

Внутренний комментарий

Выберите "Сообщение клиенту", чтобы клиент увидел комментарий

ОТПРАВИТЬ

Комментариев нет

Рисунок 4.12 – Блок комментариев с формой добавления

Для заказчика в блоке комментариев отображаются только сообщения с типом 'client'. При наличии запроса на продление срока в блоке информации появляется соответствующая отметка и кнопки для согласования или отклонения.

← НАЗАД

ЗАЯВКА #13

СОГЛАСОВАТЬ

ОТКЛОНИТЬ

ИНФОРМАЦИЯ О ЗАЯВКЕ

Дата:

05.01.2024

Техника:

Холодильник Model-4

Проблема:

Тестовая проблема 4

Статус:

Готова к выдаче

Завершена:

11.01.2024

Продление срока:

ОЖИДАЕТ СОГЛАСОВАНИЯ

КЛИЕНТ

Баранова Эмилия Марковна

89994563841

МАСТЕР

Мурашов Андрей Юрьевич

89535078985

ОЦЕНКА КАЧЕСТВА

ПОЛУЧИТЬ QR-КОД

КОММЕНТАРИИ (1) (только для клиента)

Петров Петр

КАЧЕСТВО

11.03.2026 18:00

Запрошено продление срока до 2026-03-12. Ожидается согласование с клиентом, не успеваем сделать

Рисунок 4.13 – Отображение запроса на продление срока для заказчика

6. Форма создания и редактирования заявки.

Форма создания и редактирования заявки доступна по URL /requests/create/ и /requests/<id>/update/ соответственно. Форма выполнена в виде карточки с заголовком, соответствующим выполняемому действию: «НОВАЯ ЗАЯВКА» или «РЕДАКТИРОВАНИЕ».

В верхней части страницы расположена кнопка «← НАЗАД» для возврата к предыдущей странице без сохранения изменений.

Форма содержит следующие поля:

- «Тип техники» – выпадающий список с предопределенными вариантами: Холодильник, Стиральная машина, Фен, Тостер, Мультиварка, Плита, Микроволновка;
- «Модель» – текстовое поле для ввода модели техники;
- «Описание проблемы» – многострочное текстовое поле для подробного

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		25

описания неисправности;

– «Клиент» – выпадающий список с пользователями, имеющими роль «Заказчик»;

– «Мастер» – выпадающий список с пользователями, имеющими роль «Мастер», необязательное поле.

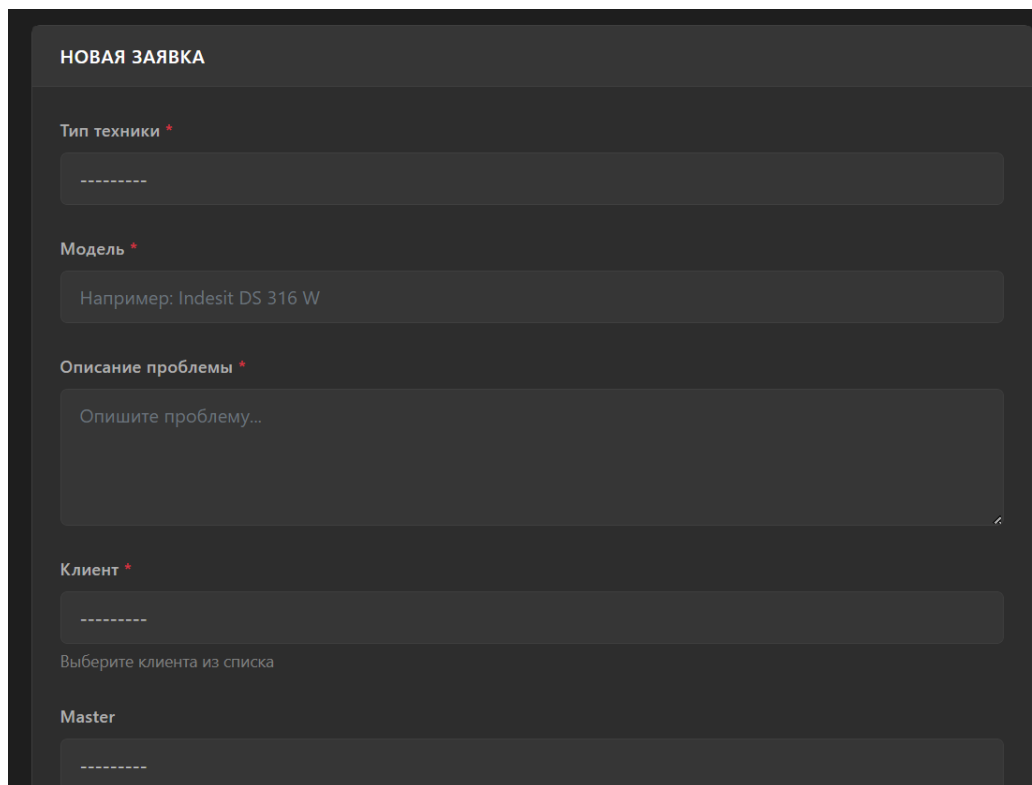


Рисунок 4.14 – Форма создания новой заявки

Под формой расположены две кнопки: «СОЗДАТЬ ЗАЯВКУ» или «СОХРАНИТЬ» синего цвета и «ОТМЕНА» серого цвета.

При попытке сохранить форму с некорректными данными поля с ошибками подсвечиваются, а под ними отображаются текстовые сообщения с пояснением, что именно нужно исправить. Для обязательных полей предусмотрена отметка красной звездочкой.

Модель *

Обязательное поле.

Например: Indesit DS 316 W

Описание проблемы *

123

Клиент *

Егорова Алиса Платоновна

Выберите клиента из списка

Master

Степанов Андрей Викторович

Можно оставить пустым, назначить позже

Дата создания будет установлена автоматически (сегодня)

СОЗДАТЬ ЗАЯВКУ

ОТМЕНА

Рисунок 4.15 – Отображение ошибок валидации формы

7. Интерфейс оператора.

Интерфейс оператора объединяет страницу списка заявок и страницу детального просмотра с расширенными правами. Оператор имеет доступ ко всем функциям управления заявками.

На странице списка заявок для оператора доступны:

- кнопка «НОВАЯ ЗАЯВКА» для создания обращения;
- полный набор кнопок действий для каждой заявки: просмотр, редактирование, удаление;
- возможность поиска заявок по фамилии клиента.

На странице детального просмотра для оператора доступны:

- кнопка «РЕДАКТИРОВАТЬ» для изменения параметров заявки;
- кнопка «УДАЛИТЬ» для удаления заявки с подтверждением;
- в блоке мастера при отсутствии назначенного исполнителя отображается форма с выпадающим списком мастеров и кнопкой «НАЗНАЧИТЬ».

Рисунок 4.16 – Форма назначения мастера для оператора

8. Интерфейс мастера.

Интерфейс мастера ориентирован на выполнение ремонтных работ и фиксацию их результатов. Мастер имеет доступ к просмотру всех заявок, но не может их редактировать или удалять.

На странице списка заявок для мастера доступна только кнопка просмотра. На странице детального просмотра мастер может:

- просматривать всю информацию о заявке;
- просматривать комментарии, оставленные другими мастерами и менеджерами;
- добавлять новые комментарии с описанием выполненных работ;
- фиксировать использованные запчасти в тексте комментариев.

ДАТА	КЛИЕНТ	ТЕХНИКА	МОДЕЛЬ	СТАТУС	МАСТЕР
05.01.2024	Баранова Эмилия Марковна	Холодильник	Model-4	Готова к выдаче	Мурашов Андрей Юрьевич
04.01.2024	Баранова Эмилия Марковна	Стиральная машина	Model-3	Готова к выдаче	Мурашов Андрей Юрьевич
03.01.2024	Баранова Эмилия Марковна	Холодильник	Model-2	Готова к выдаче	Мурашов Андрей Юрьевич
02.01.2024	Баранова Эмилия Марковна	Стиральная машина	Model-1	Готова к выдаче	Мурашов Андрей Юрьевич
01.01.2024	Баранова Эмилия Марковна	Холодильник	Model-0	Готова к выдаче	Мурашов Андрей Юрьевич

Рисунок 4.17 – Интерфейс мастера

Для добавления комментария мастер нажимает кнопку «+» в блоке комментариев, после чего открывается форма с полем для ввода текста. После отправки комментариев сразу отображается в общем списке с указанием автора и времени создания.

Мастер не видит кнопок редактирования, удаления, назначения мастеров и других административных функций.

9. Интерфейс менеджера по качеству.

Менеджер по качеству имеет расширенные права доступа, позволяющие контролировать процесс выполнения заявок и взаимодействовать с клиентами. Интерфейс менеджера по качеству включает все функции оператора, а также дополнительные возможности.

На странице детального просмотра для менеджера по качеству доступны:

- кнопка «ЗАПРОСИТЬ ПРОДЛЕНИЕ» для отправки запроса клиенту;
- в блоке мастера форма назначения исполнителя;
- при добавлении комментария возможность выбора типа: внутренний или для клиента.

— НАЗАД

ЗАЯВКА #13 ЗАПРОСИТЬ ПРОДЛЕНИЕ

ИНФОРМАЦИЯ О ЗАЯВКЕ

Дата: 05.01.2024
Техника: Холодильник Model-4
Проблема: Тестовая проблема 4
Статус: Готова к выдаче
Завершена: 11.01.2024
Продление срока: ОЖИДАЕТ СОГЛАСОВАНИЯ

КЛИЕНТ
Баранова Эмилия Марковна
89994563841

МАСТЕР
Мурашов Андрей Юрьевич
89535078985

ОЦЕНКА КАЧЕСТВА
ПОЛУЧИТЬ QR-КОД

КОММЕНТАРИИ (1) +

Петров Петр КАЧЕСТВО ДЛЯ КЛИЕНТА 11.03.2026 18:00
⚠ Запрошено продление срока до 2026-03-12. Ожидается согласование с клиентом, не успеваем сделать

Рисунок 4.18 – Интерфейс детальной заявки менеджера по качеству

При нажатии на кнопку «ЗАПРОСИТЬ ПРОДЛЕНИЕ» открывается модальное окно с формой, содержащей:

- поле для выбора новой даты завершения;
- поле для ввода комментария с объяснением причины;
- кнопка «ОТПРАВИТЬ ЗАПРОС КЛИЕНТУ».

Рисунок 4.19 – Модальное окно запроса продления срока

10. Генерация QR-кода.

Для сбора обратной связи от клиентов в системе реализована функция генерации QR-кода. В блоке «ОЦЕНКА КАЧЕСТВА» на странице детального просмотра расположена кнопка «ПОЛУЧИТЬ QR-КОД».

При нажатии на кнопку система выполняет следующие действия:

- генерирует QR-код со ссылкой на Google-форму для оценки качества;
- отображает полученное изображение на странице;
- заменяет кнопку на сгенерированный QR-код.

Рисунок 4.20 – Кнопка генерации QR-кода

QR-код содержит ссылку на внешнюю форму обратной связи, что позволяет клиентам быстро перейти к опросу, отсканировав код камерой мобильного телефона. После сканирования пользователь попадает на Google-форму, где может оценить качество обслуживания и оставить отзыв.

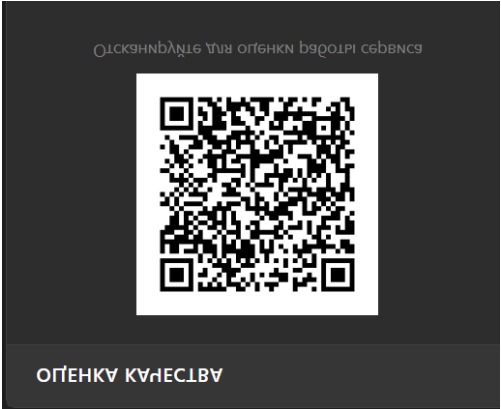


Рисунок 4.21 – Отображение сгенерированного QR-кода

5 Описание дипломного проекта

5.1 Обоснование выбора инструментальных средств разработки

Для разработки корпоративного веб-сайта и интегрированной CRM-системы для компании «Аргус» был выбран следующий технологический стек:

- Python – основной язык программирования;
- Django – фреймворк для реализации серверной логики;
- HTML/CSS/JavaScript – технологии для создания пользовательского интерфейса;
- PostgreSQL – система управления базами данных;
- Visual Studio Code – среда разработки;
- Postman – инструмент для тестирования API.

В качестве среды разработки выбран Visual Studio Code. VS Code – это расширяемый редактор кода, который позволяет легко писать, форматировать и редактировать код на разных языках. Он подсвечивает синтаксис, помогает исправлять ошибки, поддерживает отладку и работу с системами контроля версий. VS Code работает на слабых компьютерах и распространяется бесплатно.

В качестве языка программирования выбран Python. Python отличается простотой синтаксиса, высокой скоростью разработки и наличием большого количества библиотек. Он широко применяется в веб-разработке для создания серверной части приложений.

Для разработки выбран фреймворк Django. Django предоставляет встроенные компоненты для аутентификации, работы с базами данных, администрирования и маршрутизации. Это позволило сосредоточиться на бизнес-логике, а не на решении базовых задач веб-разработки.

В качестве системы управления базами данных используется PostgreSQL. PostgreSQL обеспечивает высокую надежность, целостность данных и соответствие стандартам ACID. Она применяется в системах с высокими требованиями к надежности данных и масштабируемости.

Для тестирования взаимодействия с сервером используется Postman, позволяющий проверять корректность работы API и отправлять различные типы HTTP-запросов.

Выбранный стек технологий позволил эффективно разработать надежный и функциональный корпоративный сайт с интегрированной CRM-системой для компании «Аргус».

5.2 Структура приложения

Проект представляет собой комплексное решение, состоящее из публичного корпоративного веб-сайта для клиентов и соискателей и внутренней CRM-системы

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		32

для сотрудников компании. Данные о клиентах, заявках, кандидатах, проектах, услугах и промо-материалах хранятся в единой базе данных и обрабатываются серверной частью приложения с разграничением прав доступа.

Архитектура системы включает следующие компоненты:

1. Публичный корпоративный сайт– веб-интерфейс, доступный всем пользователям, реализованный с использованием HTML, CSS и JavaScript, предназначенный для информирования клиентов и сбора заявок.

2. Внутренняя CRM-система – защищенный раздел для сотрудников компании, реализующий управление заявками, клиентами, кандидатами и контентом сайта.

3. Backend сервер (Django) – серверная часть приложения, реализующая бизнес-логику, обработку запросов, аутентификацию, разграничение прав доступа и взаимодействие с базой данных.

4. База данных (PostgreSQL) – централизованное хранение всей информации о пользователях, клиентах, заявках, кандидатах, вакансиях, проектах, услугах и промо-блоках.

Публичный корпоративный сайт включает следующие разделы:

1. главная страница – содержит промо-блок, который обновляется через базу данных, краткую информацию о компании, превью услуг, превью реализованных проектов и кнопку для отправки заявки на коммерческое предложение.

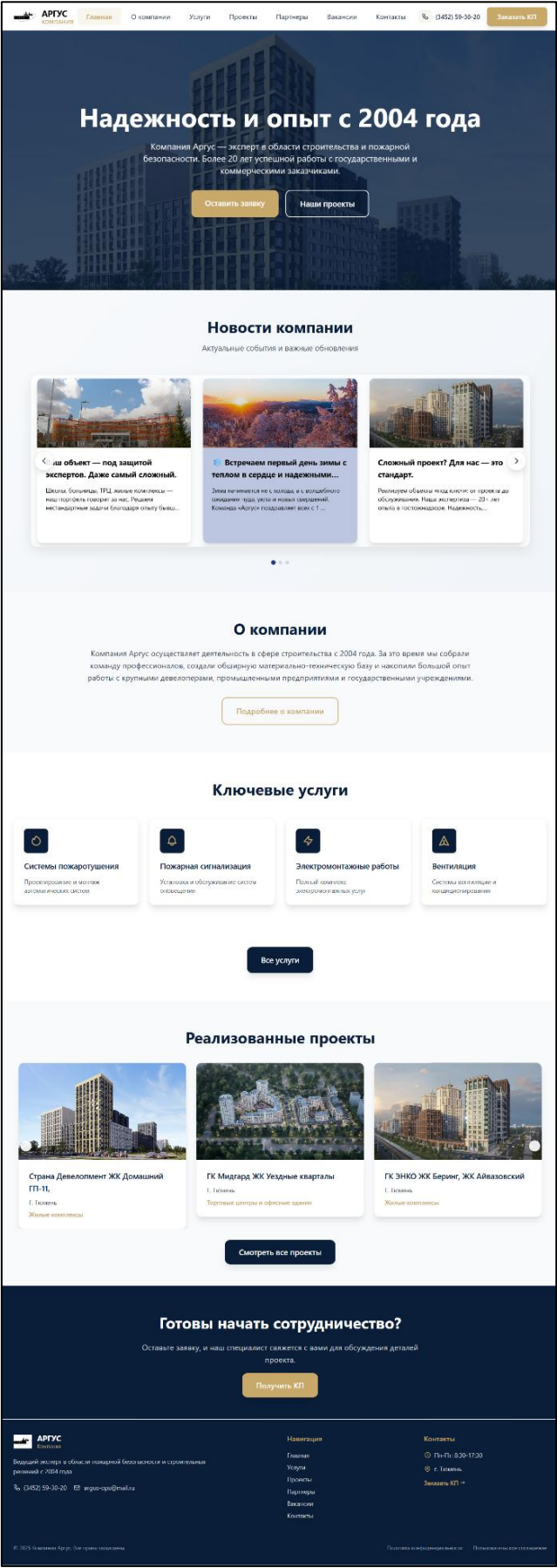


Рисунок 5.1 – Главная страница сайта

					Лист 09.02.07.000023 П.218 ПЗ 34
Изм.	Лист	№ докум.	Подпись	Дата	

2. Раздел «О компании» – история развития компании, ключевые преимущества, география деятельности и структура компании.



Рисунок 5.2 – Страница «О компании»

3. Раздел «Услуги» – каталог всех направлений деятельности компании с детальным описанием, техническими параметрами и сферой применения каждой услуги.

					09.02.07.000023 П.218 ПЗ	Лист
						35
Изм.	Лист	№ докум.	Подпись	Дата		

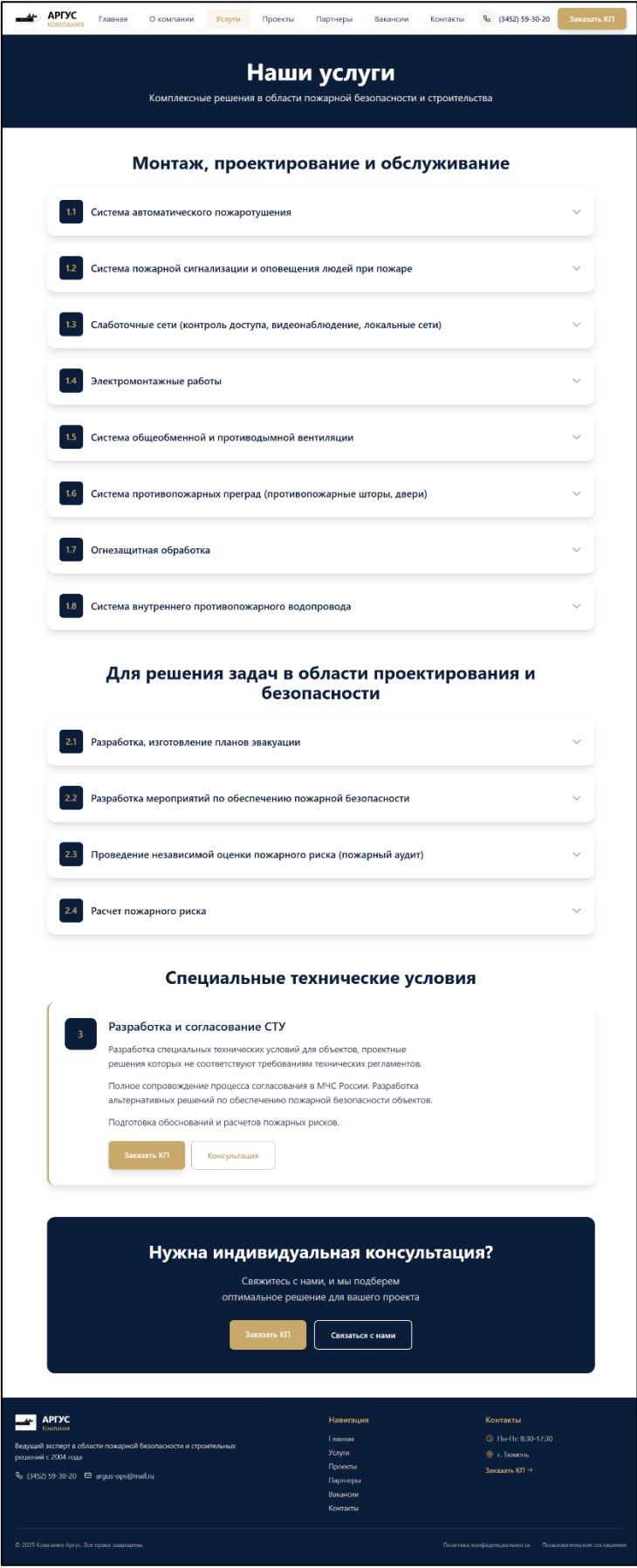


Рисунок 5.3 – Страница «Услуги»

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		36

4. Раздел «Проекты» – портфолио выполненных работ в виде сетки карточек с фильтрацией по категориям объектов (жилые, коммерческие, промышленные). Каждый проект содержит фотографии, тип объекта и географию работ.

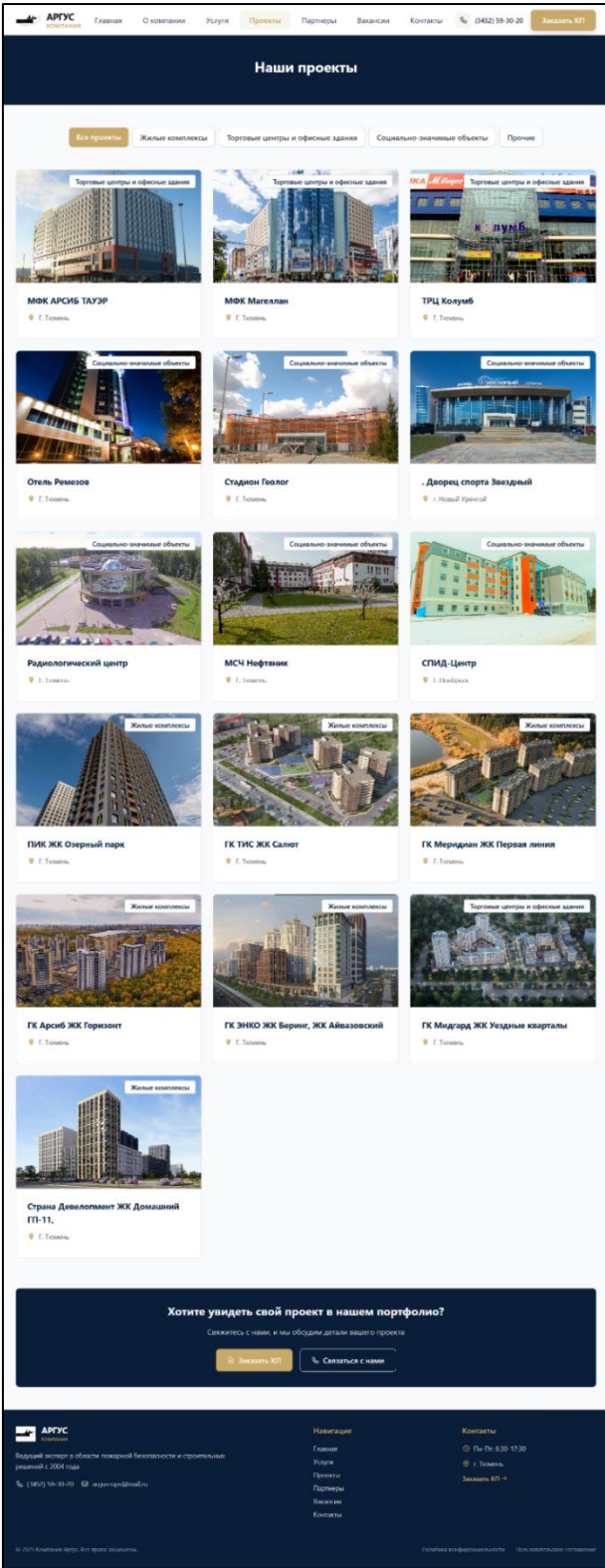


Рисунок 5.4 – Страница «Проекты»

5. Раздел «Партнёры» – разделен на две секции: «Наши заказчики» и «Наши

партнёры» с логотипами компаний, преимуществами сотрудничества и формой для связи.

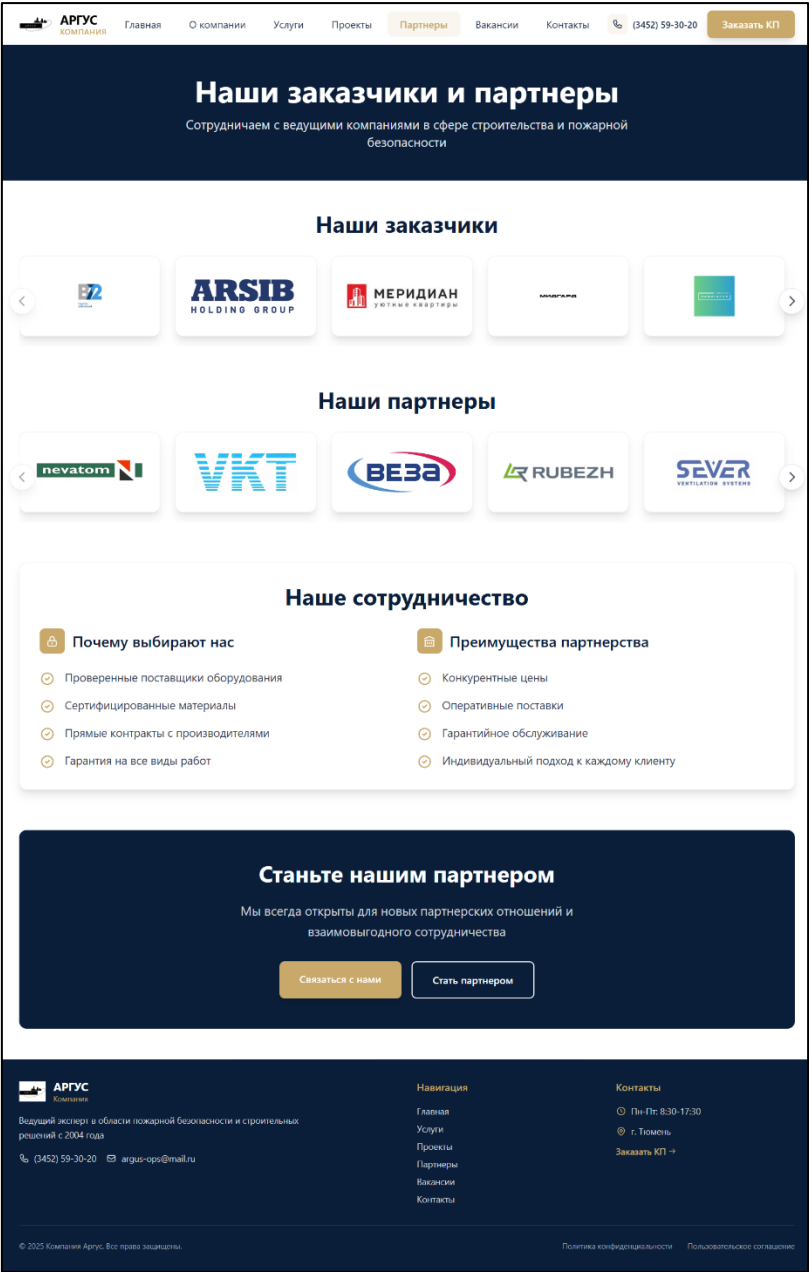


Рисунок 5.5 – Страница «Партнеры»

6. Раздел «Контакты» – контактная информация, график работы, форма обратной связи, встроенная карта местоположения.

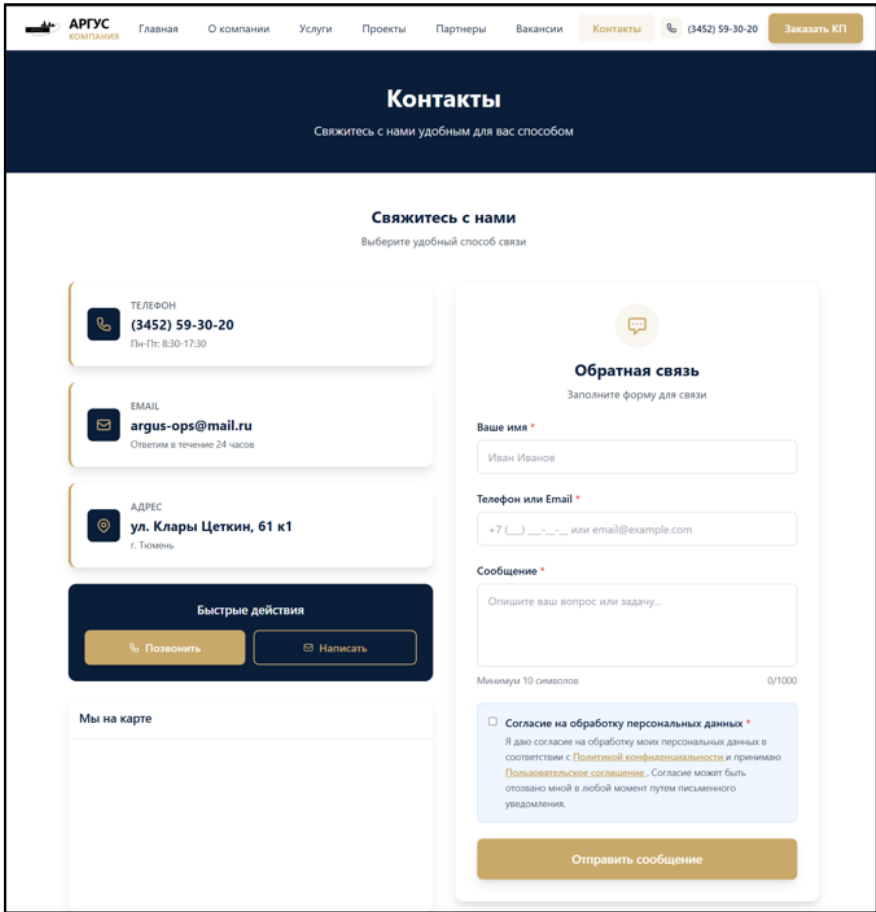


Рисунок 5.6 – Страница «Контакты»

7. Раздел «Вакансии» – список актуальных вакансий с детальными страницами, содержащими требования, обязанности и условия работы.

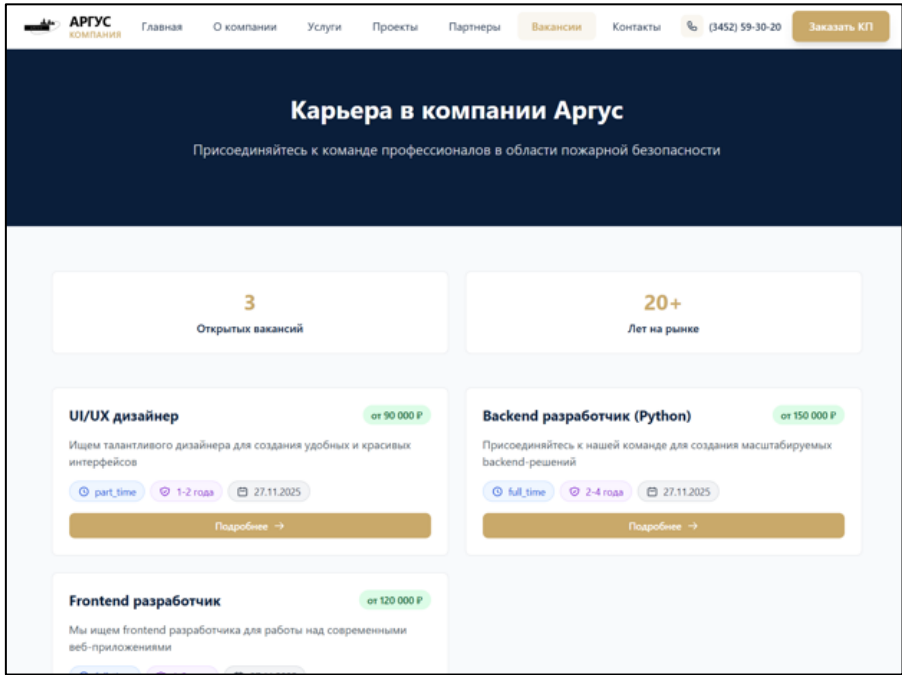


Рисунок 5.7 – Страница «Вакансии»

8. Детальная страница вакансии – подробное описание конкретной вакансии с формой отклика, содержащей поля для ввода контактных данных и обязательным согласием на обработку персональных данных.

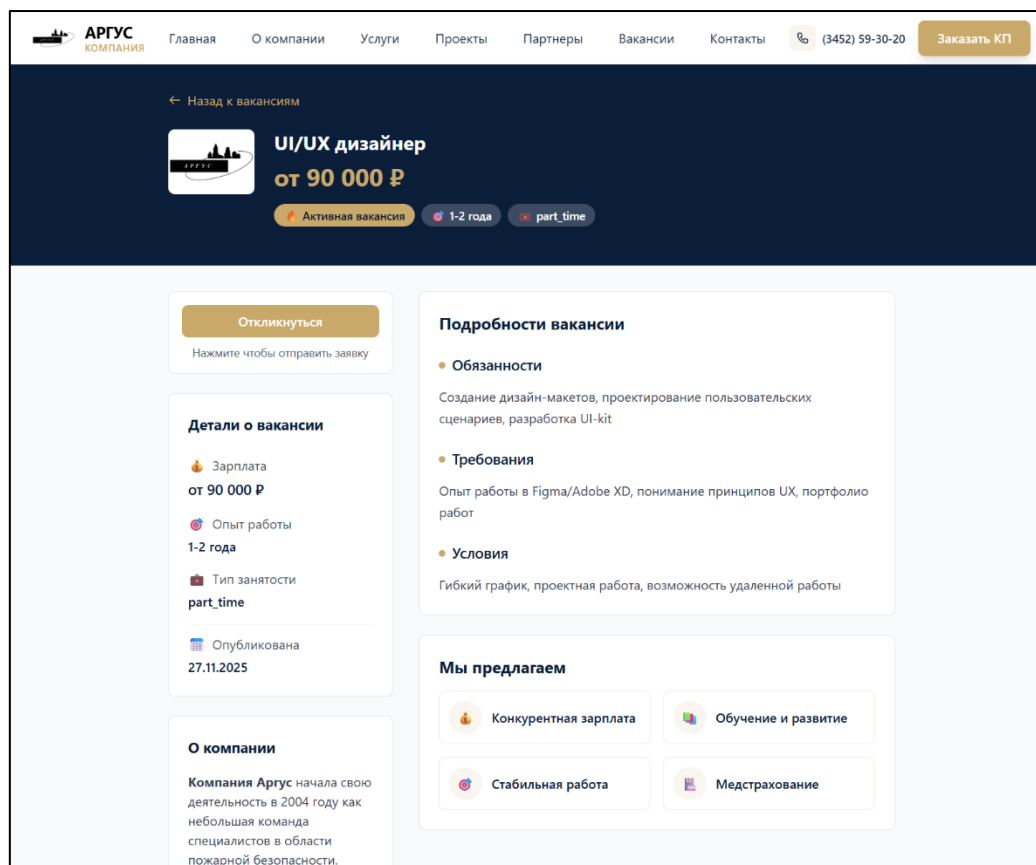


Рисунок 5.8 – Детальная страница вакансии

9. Форма заказа коммерческого предложения – многошаговая форма с полями для ввода данных компании, описания задачи, выбора услуг, прикрепления документов и обязательным согласием на обработку персональных данных.

АРГУС

КОМПАНИЯ

[Главная](#)[О компании](#)[Услуги](#)[Проекты](#)[Партнеры](#)[Вакансии](#)[Контакты](#)

(3452) 59-30-20

Заказать КП

Заказать коммерческое предложение

Заполните форму ниже, чтобы получить индивидуальное КП. Поля, отмеченные *, обязательны для заполнения.

Название компании *

ООО Пример

Контактное лицо *

Иван Иванов

Телефон *

+7 (999) 123-45-67

Email *

example@company.ru

Тип объекта *

Выберите тип объекта

Адрес объекта

г. Казань, ул. Ленина, д. 10

0/200 символов

Описание задачи

Опишите объект, особенности, пожелания...

0/1000 символов

Выберите услуги *

☐ разработка, изготовление планов эвакуации

☐ проведение независимой оценки пожарного риска (пожарный аудит)

☐ разработка мероприятий по обеспечению пожарной безопасности

☐ расчет пожарного риска

☐ Специальных технических условий (СТУ) разработка и согласование

Прикрепить документ

Нажмите для выбора или перетащите файл сюда

PDF, DOC, XLS, JPG, PNG до 10MB

☐ **Согласие на обработку персональных данных ***

Я даю согласие на обработку моих персональных данных в соответствии с [Политикой конфиденциальности](#) и принимаю [Пользовательское соглашение](#). Согласие может быть отозвано мной в любой момент путем письменного уведомления.

Заполнено форм:

0%

Отправить заявку

АРГУС

Компания

Ведущий эксперт в области пожарной безопасности и строительных решений с 2004 года

(3452) 59-30-20

argus-crp@mail.ru

Навигация

[Главная](#)[Услуги](#)[Проекты](#)[Партнеры](#)[Вакансии](#)[Контакты](#)

Контакты

Пн-Пт: 8:30-17:30

г. Тюмень

Заказать КП →

© 2025 Компания Аргус. Все права защищены.

[Политика конфиденциальности](#)[Пользовательское соглашение](#)

Рисунок 5.9 – Форма заказа КП

Внутренняя CRM-система реализует двухуровневую ролевую модель: администратор (владелец):

- полный доступ ко всем разделам системы;
- управление менеджерами (добавление, редактирование, блокировка, сброс паролей);
- управление проектами (создание, редактирование, удаление);

- управление услугами (настройка включения в КП, удаление);
- управление промо-блоками (создание, настройка активности);
- управление вакансиями (создание, редактирование, изменение статусов);
- просмотр и обработка кандидатов;
- доступ ко всем заявкам и клиентам.

Менеджер:

- доступ к работе с заявками;
- доступ к клиентской базе;
- просмотр назначенных заявок в разделе «Мои заявки»;
- обработка поступающих обращений;
- ведение клиентской базы;
- отсутствие доступа к административным разделам.

Основные функциональные модули CRM:

1. главная страница (Dashboard) – отображение всех заявок с возможностью фильтрации по статусам.

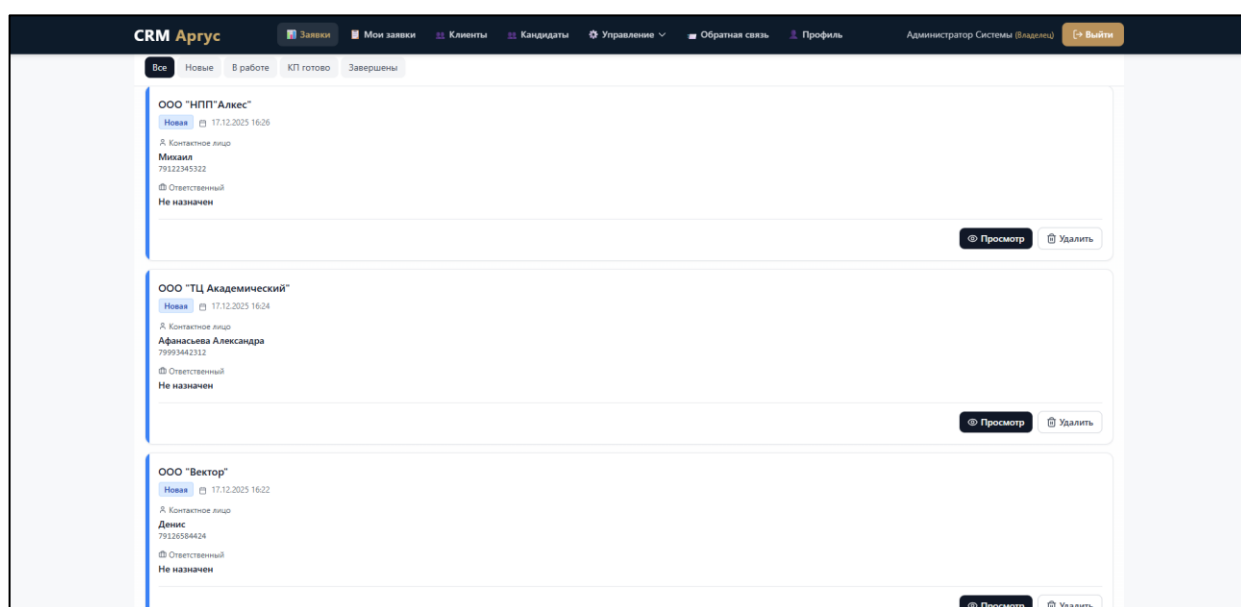


Рисунок 5.10 – Главная страница CRM

2. Модуль «Мои заявки» – отображение только тех заявок, которые назначены на текущего пользователя, с автоматической пометкой как просмотренных и пагинацией.

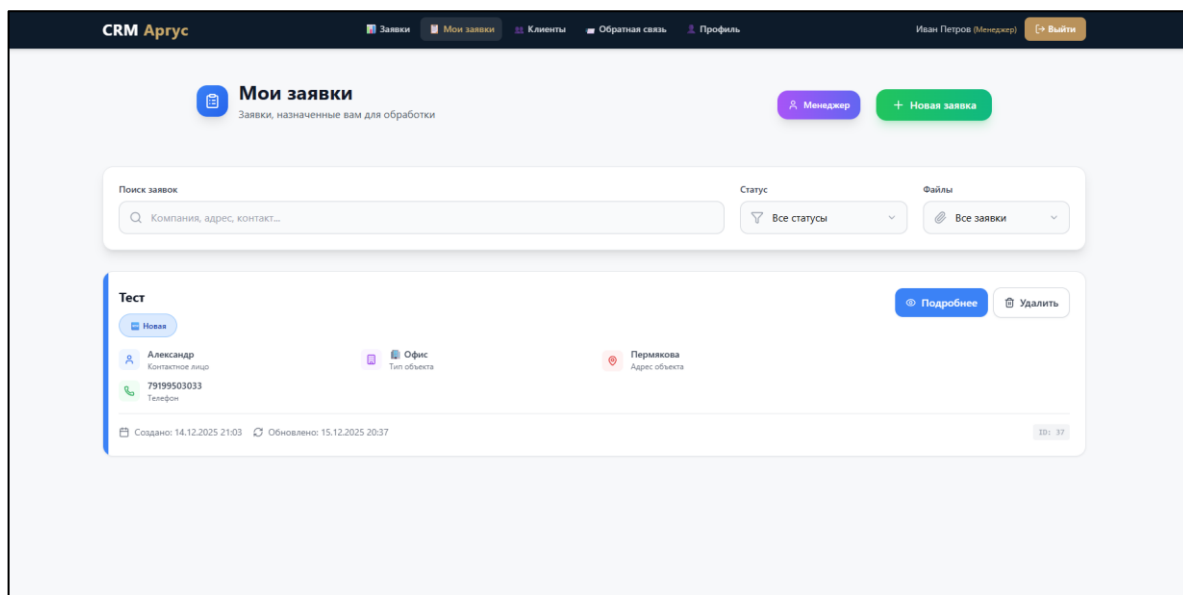


Рисунок 5.11 – Страница «Мои заявки»

3. Карточка заявки – детальный просмотр информации о заявке, включающий полные данные о клиенте, компании и объекте, историю изменения статусов, систему комментариев для внутреннего общения, возможность назначения ответственного менеджера и просмотр прикрепленных файлов.

Заявка #75 от пример

Адрес: Пермякова
 Тип объекта: Коммерческое помещение
 Дата создания: 18.12.2025 01:49
 Клиент: Александр • 79199503033
 Email: afanaseva.sasha.a@gmail.com
 Компания: пример

Ответственный менеджер:
 Не назначен

Файл не прикреплен.

Услуги:

- проведение независимой оценки пожарного риска (пожарный аудит)

Ответственный менеджер
 — Не назначен —

Статус заявки
 Новая

Добавить комментарий
 Введите комментарий...

Сохранить изменения

Комментарии

Рисунок 5.12 – Карточка заявки

4. Модуль «Клиенты» – единая клиентская база с поиском по названию компании, контактному лицу, телефону или email, карточка клиента с историей всех его заявок и автоматическим предотвращением дублирования.

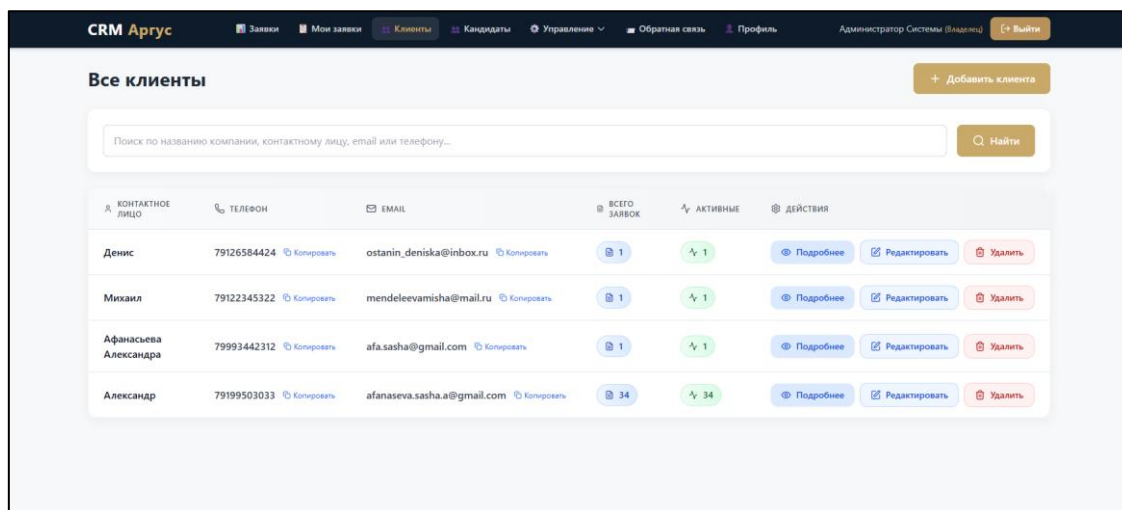


Рисунок 5.13 – Страница «Клиенты»

5. Модуль «Кандидаты» – управление кандидатами на вакансии компании, просмотр резюме и контактной информации, добавление заметок, фильтрация по вакансиям (доступно только администратору).

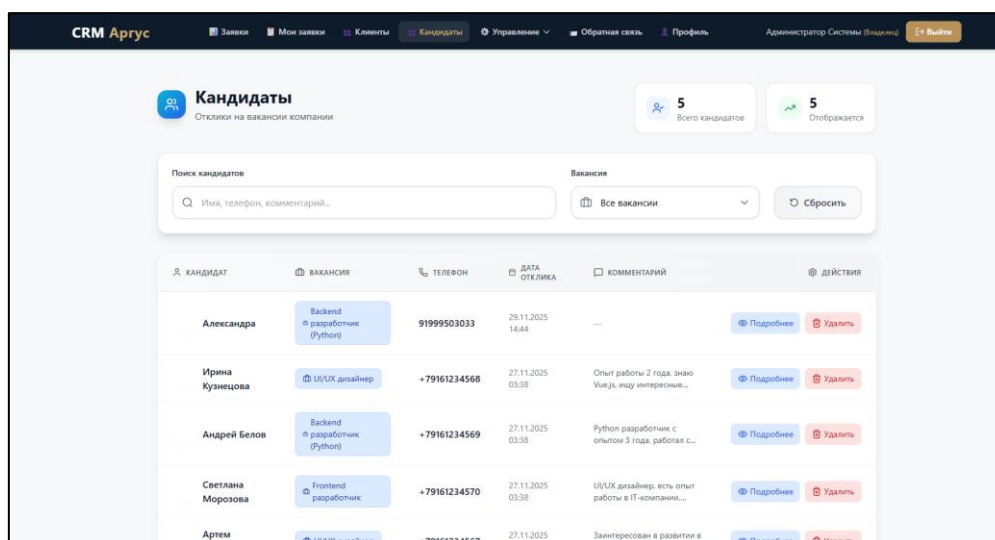


Рисунок 5.14 – Страница «Кандидаты»

6. Административный модуль «Управление – Вакансии» – создание и редактирование вакансий, управление статусами, задание требований и условий работы.

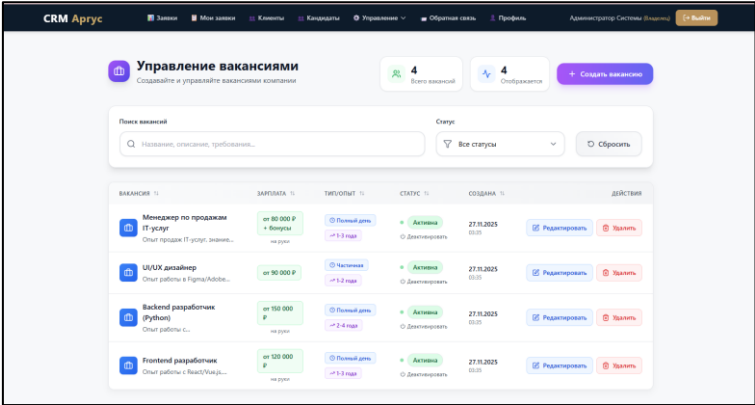


Рисунок 5.15 – Управление вакансиями

7. Административный модуль «Управление – Менеджеры» – полное управление пользователями системы: добавление новых менеджеров, редактирование данных существующих, блокировка аккаунтов, сброс паролей.

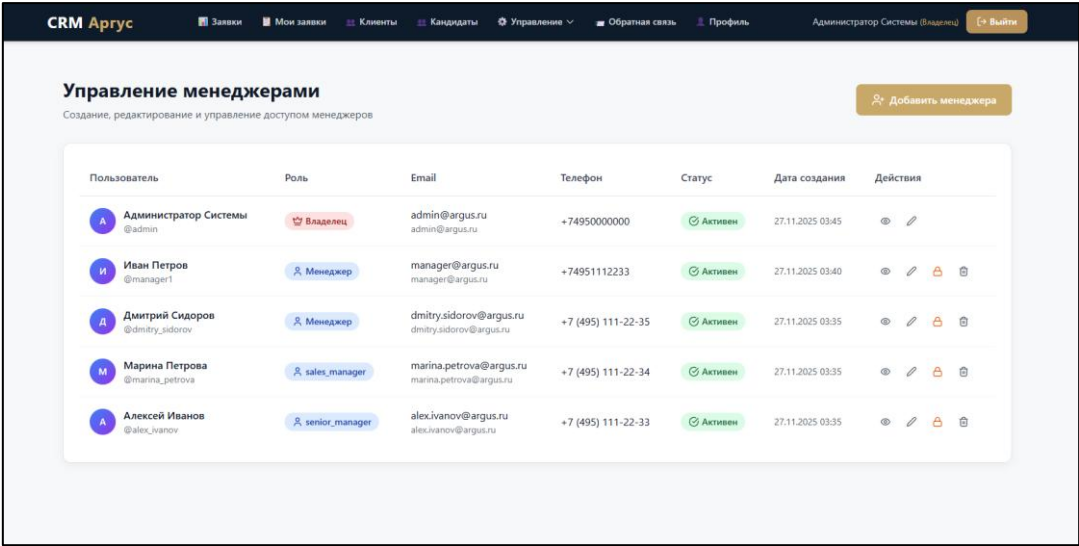


Рисунок 5.16 – Управление менеджерами

8. Административный модуль «Управление – Проекты» – ведение портфолио выполненных проектов компании с загрузкой изображений, категоризацией по типам объектов, редактированием и удалением записей.

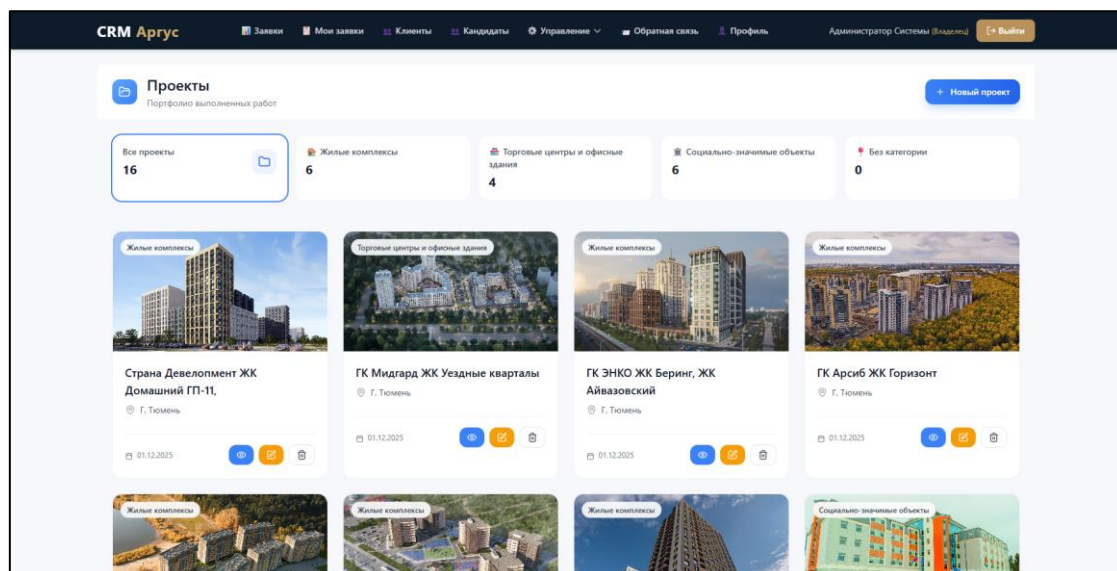


Рисунок 5.17 – Управление проектами

9. Административный модуль «Управление – Услуги» – настройка включения услуг в коммерческие предложения, удаление неиспользуемых услуг.

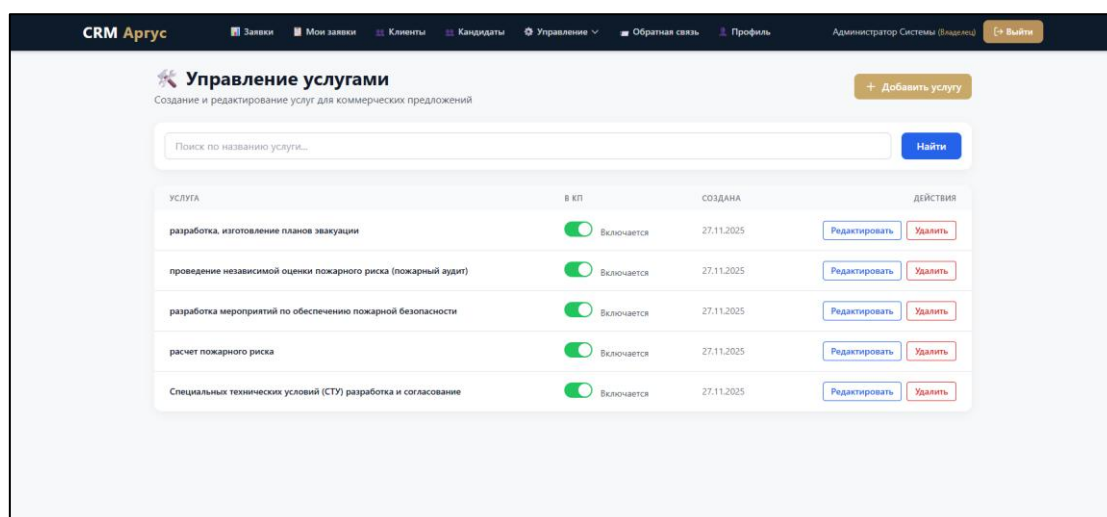


Рисунок 5.18 – Управление услугами

10. Административный модуль «Управление – Промо-блоки» – создание и управление рекламными блоками для сайта, настройка дизайна, активности и предпросмотр перед публикацией.

Изм.	Лист	№ докум.	Подпись	Дата

09.02.07.000023 П.218 ПЗ

Лист

46

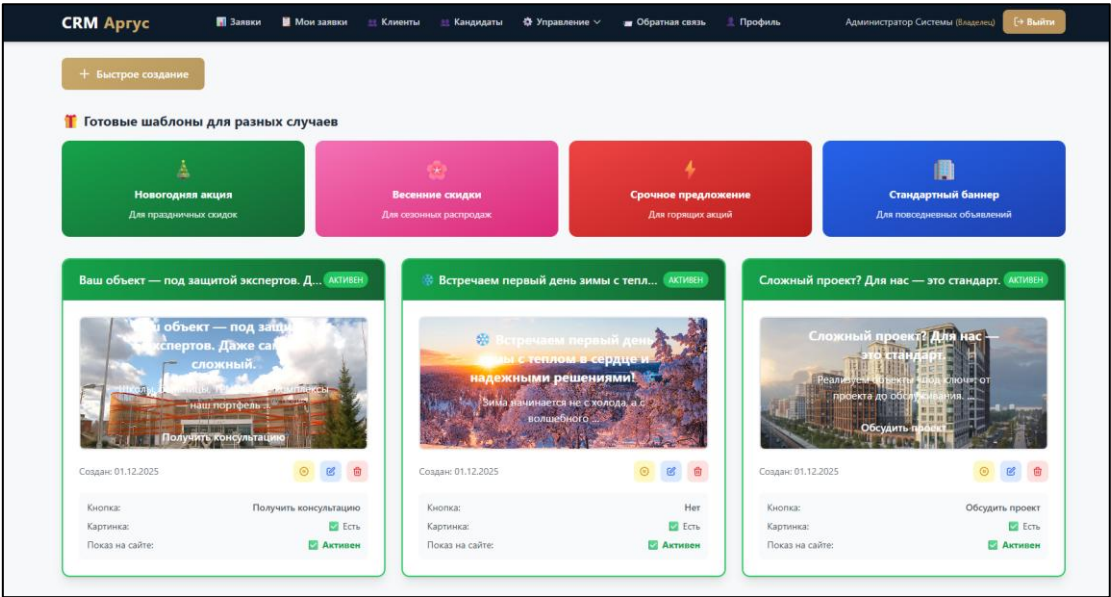


Рисунок 5.19 – Управление промо-блоками

11. Модуль «Обратная связь» – обработка сообщений, поступающих с сайта компании, фильтрация по статусам (новое, обработано, спам), изменение статусов обработки.

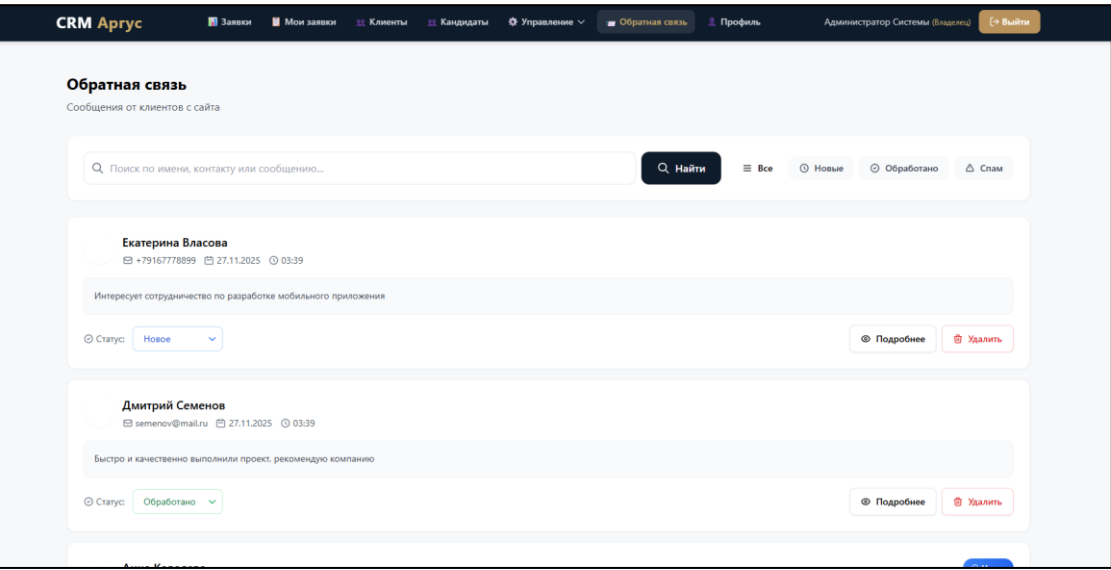


Рисунок 5.20 – Вкладка «Обратная связь»

12. Модуль «Профиль» – персональный раздел пользователя для просмотра и редактирования личных данных, смены пароля, просмотра статистики по заявкам и настройки уведомлений.

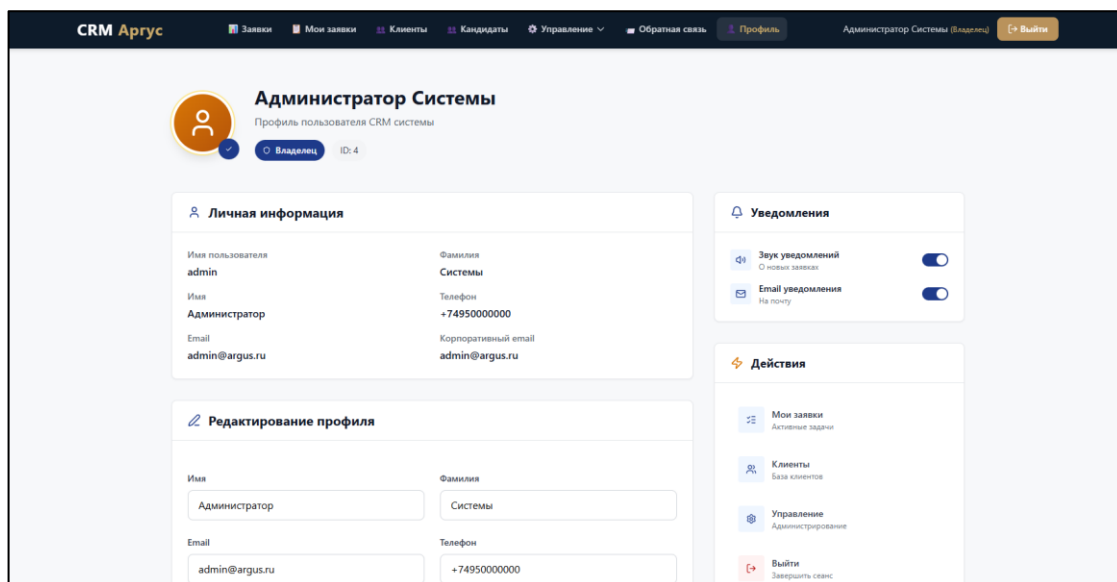


Рисунок 5.21 – Вкладка «Профиль»

13. Система уведомлений – многоуровневая система оповещений, включающая email-уведомления при изменении статуса заявки, внутренние уведомления в интерфейсе CRM, звуковые оповещения о новых заявках и бейдж с количеством непросмотренных заявок в навигации.

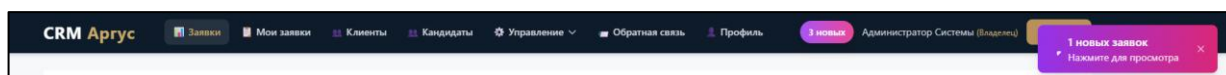


Рисунок 5.22 – Система уведомлений

14. Мобильное меню – адаптивный интерфейс с гамбургер-меню для мобильных устройств, обеспечивающий удобство работы на различных размерах экранов.

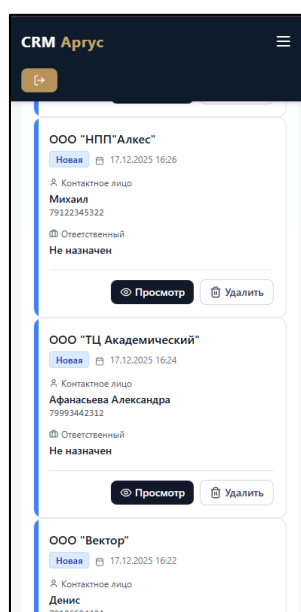


Рисунок 5.23 – Мобильное меню

Изм.	Лист	№ докум.	Подпись	Дата

09.02.07.000023 П.218 ПЗ

Лист

48

5.3 Интеграция сайта с CRM-системой

Сайт компании «Аргус» интегрирован с CRM-системой через двусторонний обмен данными. Интеграция работает в обоих направлениях: данные с сайта отправляются в CRM для обработки заявок, а CRM предоставляет контент для отображения на сайте.

Когда пользователь заполняет форму заказа коммерческого предложения или откликается на вакансию, данные не только сохраняются в базе данных сайта, но и автоматически отправляются в CRM для дальнейшей обработки менеджерами. Процесс отправки заявки включает валидацию данных на стороне сервера, сохранение в локальную базу данных, отправку в CRM через API и получение подтверждения.

Промо-блоки на главной странице управляются через CRM систему. Маркетологи могут создавать и настраивать промо-материалы без необходимости изменять код сайта.

Вакансии публикуются в CRM системе, а затем автоматически появляются на сайте. Это позволяет HR-менеджерам управлять вакансиями через единый интерфейс CRM.

Проекты компании добавляются в CRM систему и автоматически появляются в портфолио на сайте. Это обеспечивает актуальность информации о выполненных работах.

Сообщения обратной связи с сайта также отправляются в CRM для своевременного реагирования службы поддержки.

Интеграция сайта с CRM-системой обеспечивает:

- автоматизацию процессов обработки заявок;
- централизованное управление контентом сайта;
- синхронизацию данных между публичным сайтом и внутренней системой;
- актуальность информации на всех ресурсах компании.

Заключение

За время учебной практики я разработала веб-приложение для сервисного центра «БытСервис», которое помогает автоматизировать учет заявок на ремонт бытовой техники.

Я создала приложение на Django с базой данных SQLite. В нем есть шесть ролей пользователей: Администратор, Оператор, Мастер, Менеджер, Менеджер по качеству и Заказчик. У каждой роли свои права – например, операторы могут создавать и редактировать заявки, мастера добавляют комментарии по ремонту, а заказчики видят только свои заявки.

По дополнительным требованиям я добавила менеджера по качеству, который может продлевать сроки ремонта с согласия клиента, и сделала генерацию QR-кода для сбора отзывов через Google-форму.

Интерфейс получился в темном стиле, с понятной навигацией, всплывающими подсказками и модальными окнами для подтверждения удаления. Все статусы заявок выделены цветом, чтобы сразу было понятно, что происходит с ремонтом.

В итоге получилась работающая система, которая поможет сотрудникам быстрее обрабатывать заявки, не терять информацию и лучше взаимодействовать с клиентами. Задачи, которые я ставила в начале практики, выполнены, и приложение готово к использованию в сервисном центре.

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		50

Библиография

1. Django Documentation [Электронный ресурс] – Режим доступа: <https://docs.djangoproject.com/>
2. Django для профессионалов. Уайт Дж. [Электронный ресурс] – Режим доступа: <https://www.piter.com/collection/biblioteka-programmista/product/dzhego-dlya-professionalov>
3. PostgreSQL Documentation [Электронный ресурс] – Режим доступа: <https://www.postgresql.org/docs/>
4. Python Documentation [Электронный ресурс] – Режим доступа: <https://docs.python.org/>
5. Разработка веб-приложений с использованием Django. Майкл Брайант [Электронный ресурс] – Режим доступа: <https://www.ozon.ru/product/razrabotka-veb-prilozheniy-s-ispolzovaniem-django-312310007/>
6. Веб-разработка на Python и Django. Полная документация [Электронный ресурс] – Режим доступа: <https://django.fun/>
7. Шаблоны проектирования корпоративных приложений. Фаулер М. [Электронный ресурс] – Режим доступа: <https://www.piter.com/collection/biblioteka-programmista/product/shablony-proektirovaniya-korporativnyh-prilozheniy>
8. Корпоративные веб-сайты: проектирование, разработка, поддержка. А.В. Иванов [Электронный ресурс] – Режим доступа: <https://www.litres.ru/book/a-v-ivanov/korporativnye-veb-sayty-proektirovanie-razrabotka-podderzhka-115666715/>
9. Интеграция веб-приложений с CRM системами. Практическое руководство [Электронный ресурс] – Режим доступа: <https://habr.com/ru/articles/753420/>
10. Разработка пользовательских интерфейсов. А. Лебедев [Электронный ресурс] – Режим доступа: <https://www.artlebedev.ru/books/>

Приложение А
(справочное)
Листинг программы

config/settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'users',
    'requests',
]

AUTH_USER_MODEL = 'users.User'

TEMPLATES = [
    {
        'BACKEND':
'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
    },
]

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

LOGIN_URL = 'login'
LOGIN_REDIRECT_URL = 'dashboard'
```

users/models.py

```
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):
    TYPE_CHOICES = [
        ('Администратор', 'Администратор'),
        ('Оператор', 'Оператор'),
        ('Мастер', 'Мастер'),
        ('Менеджер', 'Менеджер'),
        ('Менеджер по качеству', 'Менеджер по качеству'),
        ('Заказчик', 'Заказчик'),
    ]
```

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		52

```

        phone = models.CharField(max_length=20)
        user_type = models.CharField(max_length=30,
choices=TYPE_CHOICES)
        fio = models.CharField(max_length=255)

```

requests/models.py

```

from django.db import models
from users.models import User

class Request(models.Model):
    STATUS_CHOICES = [
        ('Новая заявка', 'Новая заявка'),
        ('В процессе ремонта', 'В процессе ремонта'),
        ('Готова к выдаче', 'Готова к выдаче'),
    ]

    requestID = models.AutoField(primary_key=True)
    startDate = models.DateField()
    homeTechType = models.CharField(max_length=50)
    homeTechModel = models.CharField(max_length=100)
    problemDescription = models.TextField()
    requestStatus = models.CharField(max_length=50,
default='Новая заявка')
    completionDate = models.DateField(null=True, blank=True)
    repairParts = models.TextField(blank=True, null=True)
    deadline_extended = models.BooleanField(default=False)

    master = models.ForeignKey(User, on_delete=models.SET_NULL,
null=True, blank=True)
    client = models.ForeignKey(User, on_delete=models.CASCADE)

class Comment(models.Model):
    commentID = models.AutoField(primary_key=True)
    message = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    comment_type = models.CharField(max_length=20,
default='internal')

    master = models.ForeignKey(User, on_delete=models.CASCADE)
    request = models.ForeignKey(Request,
on_delete=models.CASCADE)

```

requests/forms.py

```

from django import forms
from .models import Request, Comment
from users.models import User

class RequestForm(forms.ModelForm):
    class Meta:

```

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		53

```

        model = Request
        fields = ['homeTechType', 'homeTechModel',
'problemDescription', 'client', 'master', 'requestStatus',
'completionDate']
        widgets = {
            'problemDescription': forms.Textarea(attrs={'rows':
4, 'class': 'form-control'}),
            'homeTechType': forms.Select(attrs={'class': 'form-
control'}),
            'homeTechModel': forms.TextInput(attrs={'class':
'form-control'}),
            'client': forms.Select(attrs={'class': 'form-
control'}),
            'master': forms.Select(attrs={'class': 'form-
control'}),
            'requestStatus': forms.Select(attrs={'class': 'form-
control'}),
            'completionDate': forms.DateInput(attrs={'type':
'date', 'class': 'form-control'}),
        }

class CommentForm(forms.ModelForm):
    comment_type = forms.ChoiceField(
        choices=[('internal', 'Внутренний'), ('client', 'Для
клиента')],
        required=False
    )
    class Meta:
        model = Comment
        fields = ['message']

```

requests/views.py

```

from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import authenticate, login, logout
from django.contrib import messages
from django.db.models import Count
from datetime import date
from .models import Request, Comment
from users.models import User
from .forms import RequestForm, CommentForm

def login_view(request):
    if request.method == 'POST':
        user = authenticate(
            request,
            username=request.POST.get('username'),
            password=request.POST.get('password')
        )
        if user:
            login(request, user)
            return redirect('dashboard')

```

```

        else:
            messages.error(request, 'Неверный логин или пароль')
            return render(request, 'requests/login.html')

    def dashboard(request):
        if request.user.user_type == 'Заказчик':
            my_requests = Request.objects.filter(client=request.user)
            return render(request, 'requests/dashboard.html',
                {'my_requests': my_requests})
        else:
            recent = Request.objects.order_by('-startDate')[:10]
            completed = Request.objects.filter(
                requestStatus='Готова к выдаче',
                completionDate__isnull=False
            )
            total_days = 0
            for req in completed:
                total_days += (req.completionDate -
                    req.startDate).days
            avg_time = round(total_days / completed.count(), 1) if
                completed.count() > 0 else 0

            tech_stats = Request.objects.values('homeTechType').annotate(
                count=Count('requestID')
            ).order_by('-count')[:5]

            return render(request, 'requests/dashboard.html', {
                'recent_requests': recent,
                'avg_repair_time': avg_time,
                'tech_stats': tech_stats
            })

    def request_detail(request, pk):
        request_obj = get_object_or_404(Request, pk=pk)
        if request.user.user_type == 'Заказчик' and
            request_obj.client != request.user:
            messages.error(request, 'Нет доступа')
            return redirect('dashboard')

        if request.user.user_type == 'Заказчик':
            comments = request_obj.comments.filter(comment_type='client')
        else:
            comments = request_obj.comments.all()

        if request.method == 'POST' and request.user.user_type in
            ['Мастер', 'Менеджер по качеству']:
            form = CommentForm(request.POST)
            if form.is_valid():
                comment = form.save(commit=False)
                comment.master = request.user

```

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		55

```

        comment.request = request_obj
        comment.save()
        return redirect('request_detail', pk=pk)

    return render(request, 'requests/request_detail.html', {
        'request_obj': request_obj,
        'comments': comments,
        'form': CommentForm()
    })

def request_create(request):
    if request.user.user_type not in ['Оператор',
    'Администратор']:
        messages.error(request, 'Нет прав')
        return redirect('dashboard')

    if request.method == 'POST':
        form = RequestForm(request.POST)
        if form.is_valid():
            new_request = form.save(commit=False)
            new_request.startDate = date.today()
            new_request.save()
            return redirect('request_detail',
pk=new_request.requestID)
        else:
            form = RequestForm()

    return render(request, 'requests/request_form.html',
{'form': form, 'title': 'Новая заявка'})

def extend_deadline(request, pk):
    if request.user.user_type != 'Менеджер по качеству':
        messages.error(request, 'Нет прав')
        return redirect('dashboard')

    request_obj = get_object_or_404(Request, pk=pk)
    request_obj.deadline_extended = True
    request_obj.save()

    Comment.objects.create(
        message='Запрошено продление срока',
        master=request.user,
        request=request_obj,
        comment_type='client'
    )
    return redirect('request_detail', pk=pk)

```

requests/urls.py

```

from django.urls import path
from . import views

```

					09.02.07.000023 П.218 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		56

```

urlpatterns = [
    path('', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('dashboard/', views.dashboard, name='dashboard'),
    path('requests/', views.request_list, name='request_list'),
    path('requests/create/', views.request_create,
name='request_create'),
    path('requests/<int:pk>/', views.request_detail,
name='request_detail'),
    path('requests/<int:pk>/update/', views.request_update,
name='request_update'),
    path('requests/<int:pk>/delete/', views.request_delete,
name='request_delete'),
    path('requests/<int:pk>/extend/', views.extend_deadline,
name='extend_deadline'),
]

```

					09.02.07.000023 П.218 ПЗ	Лист
						57
Изм.	Лист	№ докум.	Подпись	Дата		